λ_λ: A Programming Language for Silicon Photonics

Vaibhav Mehta
Cornell University
Ithaca, NY, USA

Arjun Devraj
Cornell University
Ithaca, NY, USA

Bill Owens
NYSERNet
Syracuse, NY, USA

Justin Hsu
Cornell University
Ithaca, NY, USA

Rachee Singh
Cornell University
Ithaca, NY, USA

Abstract

We present λ_λ[1], a programming language for silicon photonics. λ_λ uses a linear type system to encode the physical constraints of optics, rejecting unrealizable programs at compile time. The compiler lowers well-typed programs to a graph-based intermediate representation, then solves a constrained embedding problem to map these graphs onto arbitrary silicon photonic switch targets while minimizing signal loss. We validate λ_λ on a commercial photonic switch, demonstrating correct operation for circuit switching, time-varying rotor switching and analog in-network computation. Across various hardware targets and programs, the λ_λ compiler scales to silicon photonic switches with over 100, 000 programmable elements and handles switch programs with 128 input-output pairs. Finally, we develop a synthesizer to automatically generate λ_λ programs from high-level specifications, allowing users to program photonic hardware without reasoning about optical primitives.

CCS Concepts

• **Networks → Programming interfaces**; • **Software and its engineering → Semantics**; • **Hardware → Emerging optical and photonic technologies**.

Keywords

Photonics, Programming Languages, Optical Computing, Networking

ACM Reference Format:
Vaibhav Mehta, Arjun Devraj, Bill Owens, Justin Hsu, and Rachee Singh. 2026. λ_λ: A Programming Language for Silicon Photonics. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26), August 17–21, 2026, Denver, CO, USA.* ACM, New York, NY, USA, 25 pages.

1 Introduction

Large-scale machine learning workloads are highly sensitive to communication delays. As a result, datacenter fabrics must continue to scale bandwidth and reduce network latency. The traditional response to these needs has been to scale electrical networks along familiar axes, *i.e.,* using higher-radix switches and deeper Clos topologies [11, 22, 45, 58]. These techniques have been remarkably successful in the past, but they are reaching diminishing returns since high-radix switching ASICs are expensive and power-hungry, and additional stages in datacenter topology increase cost and operational complexity while exacerbating tail latency [34].

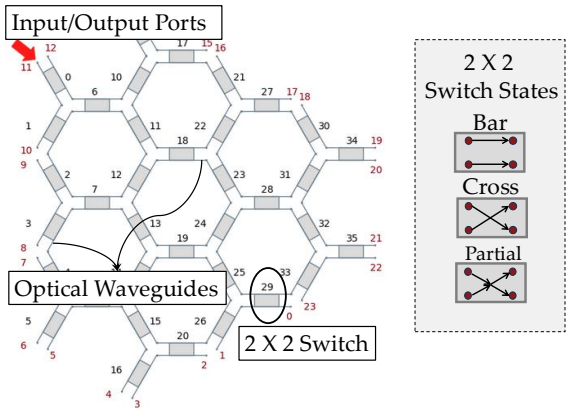


Figure 1: An example programmable photonic circuit or mesh. Waveguides (shown with lines) carry optical signals from external ports (shown with red numbers). Shaded rectangles are configurable 2×2 switching elements.

These pressures are motivating a push toward optical switching inside the datacenter [3, 16, 20, 23, 37, 46, 55]. Optical circuit switches (OCSes) provide high-bandwidth connectivity while consuming less energy, and are now deployed in production-scale ML datacenters [30, 59]. At the same time, optical switching is being reshaped by an important technology trend: silicon photonics [9, 12, 39]. By integrating optical components into photonic integrated circuits (PICs) fabricated at scale in foundries, silicon photonic technologies offer the bandwidth and energy advantages of optics with the manufacturing scalability of modern electronic chips [24, 25, 27, 29, 35].

The opportunity of PICs, however, is broader than replacing electrical packet switches with optical ones. PICs have been shown to support a range of functions beyond "vanilla" circuit switching [14, 16, 20, 23, 44, 55]. They can be used for topology engineering to improve performance and resilience of ML workloads in datacenters [50, 51]. Recent work has shown that PICs can even support forms of in-network analog computation [13, 42]. This suggests a shift in how we should view photonic switches in datacenters not only as enablers of high-bandwidth interconnects, but as programmable substrates that can transform properties of optical signals (*e.g.,* routes, power, and phase) to achieve higher-level intents.

[1]Pronounced "lambda lambda". One λ refers to the λ-calculus and the other refers to an optical wavelength.

This work is licensed under a Creative Commons Attribution 4.0 International License.
SIGCOMM '26, Denver, CO, USA
© 2026 Copyright held by the owner/author(s).

https://doi.org/10.1145/3789240.3829151

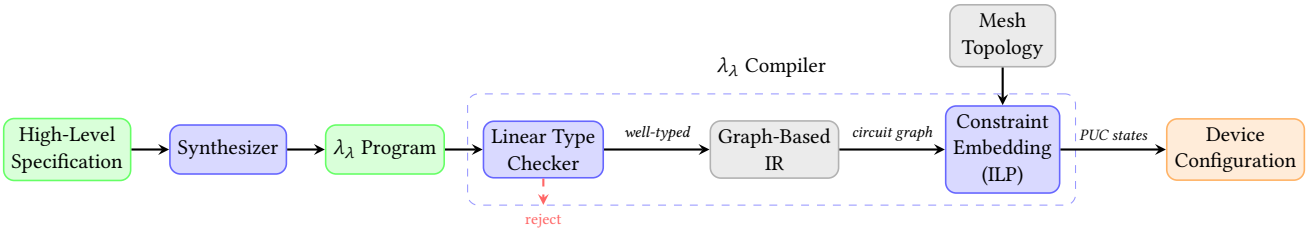


Figure 2: System architecture of λ_λ. Users provide high-level specifications or write λ_λ programs directly. The compiler type-checks programs against linear resource constraints, lowers them to a graph IR, and solves a constraint embedding problem to map the circuit onto a target mesh topology.

A key obstacle is that today's silicon photonic switches are difficult to program. Figure 1 shows a representative photonic integrated circuit in a commercial optical switch [28]. To realize even a simple intent in this circuit, programmers must manually lay out routes through the mesh of optical waveguides, configure many individual switching elements, and then verify that the resulting configuration is physically realizable due to the numerous constraints on the propagation of light. These constraints are subtle and unforgiving: optical signals cannot be duplicated arbitrarily, signals of the same wavelength cannot safely share waveguide segments, and a single port cannot be driven by multiple sources of data. As mesh size grows and programs become more complex, this low-level workflow becomes tedious, error-prone, and ultimately intractable [14].

At its core, this is a problem arising from the absence of programming abstractions for photonic switches. Networking research and practice has repeatedly benefited from moving the abstraction boundary. Instead of forcing systems to encode their intent in device-specific configuration knobs, we define a programming model that captures *what* the network should do, and we compile that intent into *how* the hardware must be configured. Programmable data planes and languages like P4 exemplify this change for electrical packet switches [7]. We argue that photonic switching is ready for similar innovation. If photonic fabrics are to become a routine part of network design, they must be programmable in a way that is (i) expressive enough to capture emerging photonic functionality, and (ii) constrained enough to guarantee physical realizability on photonic switch hardware.

In this paper, we present λ_λ, the first programming language and compiler for silicon photonic switches (Figure 2). A λ_λ program describes how optical signals should be routed and transformed. For example, "consume these inputs, apply this sequence of optical primitives, and deliver the results to these outputs," allowing a programmer to express photonic computation as well as photonic switching intents while abstracting away the device-specific details of the photonic mesh hardware. Prior work on automatically configuring silicon photonic circuits treats mesh configuration as a pathfinding problem by only routing signals through a fabric. In contrast, λ_λ develops abstractions to express how a photonic circuit should transform optical signals to implement switching or computation, treating pathfinding as one of the low-level concerns the compiler discharges internally.

Our key innovation is to reflect a basic physical fact in the language semantics: optical signals are resources that cannot be implicitly duplicated or aliased. λ_λ captures this with a *linear type system* [36]. Optical values and ports are linearly typed and must be used exactly once, which statically rules out programs that would require copying light or oversubscribing ports. In this way, λ_λ turns many configuration-time pitfalls into compile-time errors.

λ_λ's compiler can target arbitrary programmable photonic meshes. The compiler lowers a well-typed program into an intermediate graph representation that reflects the program's composition of optical resources and its flow of optical operations. It then maps this graph onto a concrete device by solving a constrained embedding problem. It selects routes in the waveguide mesh that respect the mesh topology, while enforcing constraints like path disjointness for the same wavelength. The output is a concrete per-element configuration for the target photonic switch that implements the program. Overall, this paper makes four contributions:

- **Language.** A core programming model for photonic switching that provides abstractions for routing and optical signal transformations (*e.g.*, phase shifting, splitting).
- **Static semantics.** A linear type system that encodes key physical constraints of optics, rejecting unrealizable programs at compile time.
- **Compilation.** A compilation pipeline that maps high-level programs to concrete configurations on arbitrary photonic meshes via constraint-based embedding, enabling portable programming across architectures.
- **Synthesis.** A synthesizer that generates photonic circuits from high-level specifications, enabling rapid prototyping and deployment.

Results. We evaluate λ_λ on photonic programs drawn from prior work: circuit switching [14, 16], time-varying rotor switching [32], optical multicast, and in-network analog computation [13, 35]. We validate the correctness of λ_λ programs by comparing compiler-generated configurations against hand-tuned configurations on a commercial photonic switch. The λ_λ compiler scales to large meshes (10^5 programmable units) and complex programs (128 input/output port pairs). More broadly, with λ_λ, we argue that as photonic devices become more capable, the limiting factor will not be the physics of switching but the software stack that makes those capabilities usable. By raising the level of abstraction and making physical correctness a property of programs rather than a property of manual configuration, λ_λ aims to make silicon photonic switches a substrate for rapid innovation in datacenters. Our code is available online [31].

2 Programmable Silicon Photonic Switching

A silicon photonic switch is a *photonic integrated circuit* (PIC) that steers and transforms optical signals using on-chip waveguides—the on-chip optical pathways that carry light—and programmable optical elements. Conceptually, it is similar to an electrical switch, but its "wiring" is a fixed *mesh* of optical waveguides, and its "configuration knobs" are the components that control how light couples, interferes, and propagates through that mesh (Figure 1).

A silicon photonic switch exposes input and output ports that connect to end hosts via optical fibers and transponders. Each transponder modulates data onto one or more optical wavelengths and sends it over fiber to an input port on the switch. Outgoing signals from the switch return via fiber from an output port. The optical switch is typically transparent in the sense that it is not performing optical-to-electrical-to-optical conversions. Instead, it routes optical signals, enabling the network-level abstraction of optical circuits between the input and output port pairs of the switch.

Within the switch, signals travel along the waveguide mesh and pass through a large number of *programmable unitary cells* (PUCs), the basic 2×2 programmable elements, often realized as a tunable coupler or Mach–Zehnder interferometer with phase shifters [39]. A PUC has two inputs and two outputs, and implements a 2×2 tunable *unitary transformation* on the optical signals. Intuitively, a unitary transformation is a transformation that preserves total optical power. Together, many 2×2 PUCs can realize larger unitary transformations by interconnecting them [10, 38]. Operationally, a PUC can be configured in one of three states: (1) **Bar:** input signals pass straight through, *i.e.,* top→top, bottom→bottom, (2) **Cross:** input signals swap paths, *i.e.,* top→bottom, bottom→top, and (3) **Partial:** a programmable transformation that can couple the powers and phases of two inputs and yield two output signals. Figure 1 shows the three states of a PUC.

We focus on 2×2 PUCs because they are the hardware-level configurable primitives exposed by many silicon photonic meshes. This does not restrict the programming model to 2×2 behaviors: larger $N \times N$ transformations are realized by composing many such cells. This differs from MEMS/LCOS-style systems, where programming the hardware does not directly expose a primitive that permits computation.

2.1 Why is it hard to program photonic switches?

To even implement a basic switch on a photonic circuit, the programmer has to specify a matching between ingress and egress ports. For each desired connection $i \rightarrow j$ in the mapping, the programmer creates an optical circuit from input port i to output port j (on wavelength λ) through the waveguide mesh. Realizing this intent on a waveguide mesh requires (i) choosing a feasible route through the waveguide mesh and (ii) configuring each traversed PUC so that light entering at i is steered hop-by-hop to j. Figure 3 shows an example of such a configuration. A complete switch configuration is the set of these routed circuits for all input/output port pairs, with full-duplex links implemented as two directed circuits, one per direction.

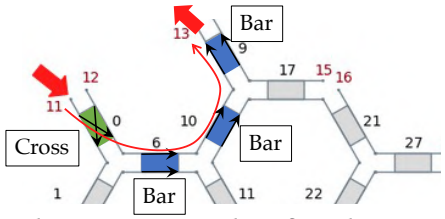


Figure 3: A photonic circuit switch configured to connect ingress port 11 to egress port 13 through the route shown. Implementing this requires configuring each PUC along the path to the appropriate bar/cross states. The greyed-out PUCs are not used in this configuration.

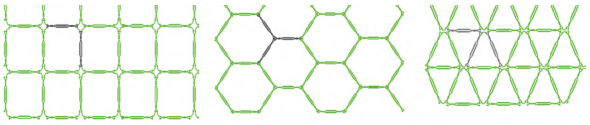


Figure 4: Programmable photonic mesh families from [8]: (a) *rectangular feedforward,* (b) *hexagonal lattice,* (c) *triangular lattice.* All can be abstracted as a graph of 2×2 programmable elements connected by waveguides as we show in §4.

Modifying switch configuration. Going from one circuit-switching configuration to another (*e.g.,* from one matching to the next) is a global update to the mesh state that may require reprogramming many PUCs to carve new paths through the shared mesh. Even when the *logical* change is simple (*e.g.,* swap which outputs two inputs reach), the *physical* change can cascade because paths contend for the same finite waveguide resources and a PUC's state simultaneously affects whatever signals traverse that site.

Correctness constraints. These configurations must also satisfy physical constraints that do not arise in electrical packet switching. First, propagation loss accumulates along a path, so feasible circuits must respect a loss budget. In practice, this couples routing to optimization, since shorter (or lower-loss) routes are preferable. Second, a circuit must preserve the wavelength it was launched on, and the configuration must be safe even when using multiplexed wavelengths on the mesh. Third, circuits must not overlap in ways that create interference or contention: a single output port cannot be driven by multiple sources, and two signals of the same wavelength cannot safely share waveguide segments. These constraints decide if a switch configuration is physically realizable on the hardware.

Many programmer intents. Circuit switching is one of the many *intents* a programmer can specify for a modern programmable photonic switch. Because PUCs can implement more than bar/cross routing, the device can realize higher-level intents that combine routing with signal transformations: *e.g.,* structured connectivity schedules, multicast-like fanout via controlled signal splitting, and analog linear operations that exploit the partial state of PUCs. Two motivating examples we use throughout the paper are *time-varying* switching (*e.g.,* , rotor-style schedules that repeatedly cycle through a sequence of matchings [32]) and *optical compute* (*e.g.,* AllReduce-like aggregation [13]).

Multiple hardware targets. Programming photonic switches is further complicated by the fact that there is no single canonical mesh architecture. Different PICs expose different waveguide topologies *e.g.,* hexagonal-, triangular-, or feedforward-style meshes

(Figure 4), different port placements, and different geometry constraints, even when they share the same conceptual building blocks. As a result, a configuration procedure that is handcrafted for one chip is brittle as it may not port to another chip, and it is difficult to maintain as devices scale in size and architectural diversity. Note that our examples (Figure 1, Figure 3) and hardware evaluation use a hexagonal mesh-based photonic switch, but our contributions apply to arbitrary mesh geometries.

2.2 Need for programming abstractions

Taken together, these observations motivate the central thesis of this paper: silicon photonic switches can express many powerful intents, but today they lack programming abstractions that separate *what* the network should do from *how* the switch must be configured. This mirrors the historical trajectory of programmable electrical switches, where early vendor-specific configuration models gave way to languages like P4 that define a portable, intent-level programming interface and rely on compilation to produce target-specific configurations.

In this work, we present λ_λ, a programming language and compiler for silicon photonic switches that achieves this separation of concerns. We view λ_λ as the "assembly language" for photonic switches: it provides control over individual programmable unit cells of the waveguide mesh while serving as a foundation for higher-level abstractions. We deliberately do not commit λ_λ to a fixed set of higher-level abstractions. Such abstractions depend critically on the assumed optical measurement model—e.g., whether computation is interpreted through intensity measurements, interferometric readout, or other detection schemes—and on device and application-specific constraints. This design allows λ_λ to serve as a common compilation target across encoding schemes and application domains, while still enforcing correctness constraints. Table 1 summarizes the analogy between P4 for electrical switches and λ_λ for photonic switches. Both languages provide abstractions for programming their respective substrates, enabling portability across diverse hardware targets while using compilers to enforce physical constraints and generate low-level configurations.

3 λ_λ Language Overview

λ_λ is a programming language that exposes the basic primitives of programmable photonic switches while guaranteeing physical realizability of circuits. A λ_λ program specifies a circuit by reading photonic signals from input ports, applying optical transformations, and routing results to output ports.

Key design principle. Optical signals and ports are linear resources *i.e.,* optical signals cannot be duplicated, and routing two signals to the same output causes interference. λ_λ captures these physical constraints through a linear type system that statically rejects programs violating optical physics—turning configuration-time errors into compile-time errors.

3.1 Syntax and Semantics

Figure 5 summarizes the core syntax of λ_λ. We introduce the language through a simple example and then explain each construct. The following program reads light from input port 0, applies a phase shift of π radians, and routes the result to output port 1:

```
e ::= v | x |  input(p) | let x = e1 in e2
    | (e1, e2) | unpack e1 as (x1, x2) in e2
    | phase(θ, e)| split(r, e)
    | unitary(U, (e1, e2))
    | output(p) <- e1; e2
v ::= r ∈ ℝ | p ∈ Port | U ∈ Unitary | ()
τ ::= ℝ |  Port | Opt | Unitary | Unit |(τ1 * τ2)
```

Figure 5: Full syntax of λ_λ. Expressions describe photonic circuits that read signals from input ports, apply optical transformations, and route results to output ports.

```
let x = input(0) in
let y = phase(π, x) in
output(1) <- y; ()
```

This program illustrates the three stages of a typical λ_λ computation: *signal creation* via `input`, *signal transformation* via `phase`, and *signal routing* via `output`. We now describe each category of constructs.

Values. λ_λ programs operate over basic value forms:

- *Real numbers* ($\mathbb{R}$): parameters for optical components like phase shifts and splitting ratios;
- *Port identifiers* (Port): names of I/O ports on the switch;
- *Photonic signals* (Opt): optical signals flowing in circuits;
- *Unitary values* (Unitary): 2×2 unitary transformations;
- the *unit value* (), which carries no information and is used when computation produces no signal, but it might have a side effect (*e.g.,* routing a signal to an output port)

Signal creation. The `input(p)` statement reads a photonic signal from input port `p`, creating a new optical signal. Each input port can be read at most once. Attempting to read the same port twice is a type error, reflecting the physical constraint that it is not feasible to have multiple independent copies of the same optical signal.

Signal transformation. λ_λ provides three operations for manipulating optical signals:

- `phase(θ, e)` changes the phase of an optical signal `e` by θ, and returns the transformed signal.
- `split(r, e)` splits an optical signal `e` with amplitude A into two signals: one with amplitude $r \cdot A$ and another with amplitude $(1 - r) \cdot A$, returning a *pair* of signals.
- `unitary(U, (e1, e2))` applies a 2×2 transformation `U` to a pair of signals, returning a pair of transformed signals.

Each transformation *consumes* its input signals *i.e.,* signals cannot be reused afterward, ensuring that the signal entering a component is absorbed and transformed, not copied.

Signal routing. The construct `output(p) <- e1; e2` routes signal e_1 to output port `p`, then continues with expression e_2. Similar to input ports, each output port can be driven by at most one signal. The final expression typically returns `()` once all signals have been routed.

Sequencing and pairs. Standard functional programming constructs are used to compose these primitives: `let x = e1 in e2` binds the result of e_1 to variable `x` for use in e_2. Pairs `(e1, e2)` group two values, and `unpack e1 as (x1, x2) in e2` deconstructs a pair for use later.

Comparison	P4 (Electrical)	λ_λ (Photonic)
Target hardware	Programmable electrical switch ASICs	Programmable photonic integrated circuits
Programmable elements	Match-action tables	PUC configurations
Program Portability	Protocol-independent (custom headers/parsers)	Topology-independent (custom meshes/geometries)
Compiler guarantees	Table sizes, pipeline stages, action constraints	Physical realizability, wavelength conflicts, loss budgets
Example intent	"Match on dst IP, forward to port 3"	"Switch ingress port 2 to egress port 5 on λ_1"

Table 1: P4 and λ_λ achieve analogous abstraction goals for their respective substrates: both separate programmer intent from hardware configuration, enable portability across targets, and use compilers to enforce physical constraints.

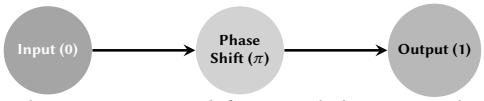


Figure 6: Photonic circuit graph for a simple λ_λ program that reads a photonic signal from input port 0, applies a phase shift of π radians, and routes the result to output port 1.

3.2 Intermediate Representation

To compile λ_λ programs onto physical PUC meshes, the compiler first lowers the source code into a graph-based *intermediate representation* (IR). This IR, called a *photonic circuit graph*, captures the structure of circuits created by the program in a form suitable for mapping onto hardware.

Figure 6 shows the photonic circuit graph for our running example λ_λ program: an input node connects to a phase node, which connects to an output node. The λ_λ compiler constructs this graph by traversing the program's abstract syntax tree (AST) and creating nodes and edges inductively based on program structure. Appendix D formalizes this procedure and gives semantics to photonic circuit graphs. We prove our compilation to the graph correct by showing that any value computed by a λ_λ program is also computed by the corresponding photonic circuit graph.

3.3 Well-Formedness and the Type System

Not every photonic circuit graph corresponds to physically realizable circuits. A *well-formed* graph must satisfy two constraints that arise from optics. First, each optical signal must be consumed exactly once, and each output port must receive exactly one signal. Second, each component must have the correct *arity*. For example, a phase shifter operates on one signal, while a 2×2 unitary operates on two.

Definition 3.1 (Well-Formed Photonic Circuit Graph). A photonic circuit graph G is *well-formed* if:

- each port number appears in at most one node.
- `input` nodes have out-degree 1.
- `output` nodes have in-degree 1.
- `phase` nodes have in-degree 1 and out-degree 1.
- `split` nodes have in-degree 1 and out-degree 2.
- `unitary` nodes have in-degree 2 and out-degree 2.

λ_λ uses a *linear type system* to ensure that every well-typed program compiles to a well-formed circuit graph. A *type system* is a set of rules for assigning a *type* to each expression in a program. The type system assigns a type (*e.g.,* $\mathbb{R}$, Opt, or a pair type) to each expression in a λ_λ program and checks these assignments statically, *i.e.,* before the program is compiled to a circuit graph. The key idea is that optical signals and ports are given *linear types*, meaning each must be used exactly once *i.e.,* a signal cannot be duplicated or silently discarded. If a program violates these constraints, the type checker rejects it as *ill-typed*. This flags errors like wiring a component with the wrong kind or number of inputs, or attempting to reuse a signal or port. We present the full details of the type system in Appendices B and C. The main guarantee of the type system is that well-typed programs compile to well-formed circuit graphs. In other words, if a program passes the type checker, then the circuit graph produced by compiling it will respect physical constraints. Formally, we state this guarantee as follows:

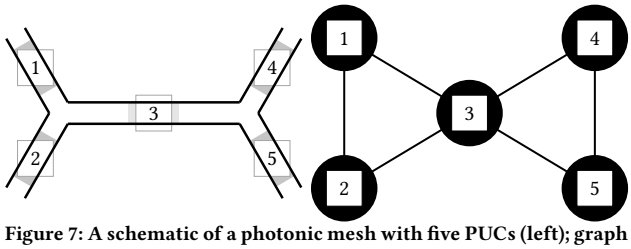


Figure 7: A schematic of a photonic mesh with five PUCs (left); graph representation of the mesh (right).

Theorem 3.2 (Well-Typed Programs Compile to Well-Formed Circuits). *If a λ_λ program e is well-typed, then the photonic circuit graph produced by compiling e is well-formed.*

The proof of this theorem is in Appendix D (Theorem D.9).

4 λ_λ Compiler

Given a photonic circuit graph IR, the compiler realizes it on a concrete photonic mesh. It (i) chooses which physical sites (PUCs and I/O ports) implement each logical operation and (ii) routes each connection in the photonic circuit graph as a path through the mesh without violating optical and geometric constraints. We formulate this as a *constrained graph embedding* problem: we embed the circuit graph into a hardware mesh while minimizing end-to-end signal loss.

Hardware model. The compiler requires a description of the target mesh. A target switch hardware consists of *programmable unit cells* (PUCs) interconnected by waveguides, as shown in Figure 7, left. We abstract this layout as a graph (Figure 7, right) where each PUC is a vertex and each waveguide segment between neighboring PUCs is an edge.

Formally, the mesh topology is a directed host graph $H = (U, A)$. Each vertex $h \in U$ represents a physical site *i.e.,* a PUC or an I/O port. Each directed edge $(i, j) \in A$ represents a directed waveguide segment from i to j, with weight $w_{ij} > 0$. The weight captures properties of the waveguide edge, *e.g.,* propagation or insertion loss along that segment. Each host h advertises the operations it can implement through a *candidate set* $T(h)$:

$$T(h) \subseteq \{\text{split}, \text{phase}, \text{unitary}, \text{input port}_i, \text{output port}_i\}.$$

Many geometry constraints depend on which physical *face* of a PUC a waveguide connects to. We capture this by associating each host h with *side groups* $S_{h,1}, \ldots, S_{h,k_h} \subseteq U$, which partition h's neighbors by face (neighbors connected to ports on the same face belong to the same group). For example, in Figure 7, host 3 has two side groups: $S_{3,left} = \{1, 2\}$ and $S_{3,right} = \{4, 5\}$.

4.1 Embedding Circuit Graphs onto Photonic Meshes

We embed a photonic circuit graph $G = (V, E, l)$ onto the host graph $H = (U, A)$. An embedding selects (i) an injective placement of each node of G onto a compatible vertex in the host graph H and (ii) a path in H for each circuit edge, subject to physical constraints. We compute this embedding using an Integer Linear Program (ILP) that chooses placements and routes while minimizing signal loss.

Candidate pruning. Before constructing the ILP, we compute a feasible host node set $C(v) \subseteq U$ for each vertex $v \in V$ in the circuit graph G. A host node h is compatible with v if (i) it can implement the required operation $l(v)$ (*i.e.,* $l(v) \in T(h)$), and (ii) it has enough incoming and outgoing edges to accommodate the in- and out-degrees of v, *e.g.,* it automatically rules out placing a 2×2 unitary transformation on the input ports of the switch. Restricting placements to $h \in C(v)$ makes the ILP smaller.

Decision variables. Let $e = (u, v) \in E$ be a directed edge in the circuit graph. We introduce the variables:

- $y_{v,h} \in \{0, 1\}$: place circuit vertex v on host $h \in C(v)$.
- $z_{h,\tau} \in \{0, 1\}$: configure host h to implement label $\tau \in T(h)$.
- $x_{e,i,j} \in \{0, 1\}$: route edge e through host arc $(i, j) \in A$.

For multi-input labeled components, we ensure that all inputs enter from the same physical face of the chosen PUC using side-selection variables $g_{v,h,s} \in \{0, 1\}$, which select a side group $s \in \{1, \ldots, k_h\}$ when placing v on h.

Constraints. We group constraints on the ILP variables into four families: (i) placement and labeling, (ii) routing, (iii) path correctness, and (iv) geometry.

(i) Placement and labeling constraints. Each circuit vertex must be placed exactly once on a compatible host node, hosts may implement at most one circuit vertex, and used hosts must be configured with a label that matches the placed vertex's label:

$$\sum_{h \in C(v)} y_{v,h} = 1 \qquad \forall v \in V \tag{1}$$

$$\sum_{v:\, h \in C(v)} y_{v,h} \le 1 \qquad \forall h \in U \tag{2}$$

$$\sum_{\tau \in T(h)} z_{h,\tau} = \sum_{v:\, h \in C(v)} y_{v,h} \qquad \forall h \in U \tag{3}$$

$$y_{v,h} \le z_{h,l(v)} \qquad \forall v \in V,\ \forall h \in C(v) \tag{4}$$

Eq. (1) chooses one host per circuit vertex. Eq. (2) enforces injectivity *i.e.,* no two operations occupy the same physical site. Eq. (3)–(4) ensure that any host used by the embedding selects exactly one configuration label and that the label matches the placed circuit vertex.

(ii) Routing constraints. Each circuit edge $e = (u, v)$ must map to a directed path in H from the chosen location of u to the chosen location of v. We encode routing using unit-flow (multi-commodity flow) constraints. For each edge $e \in E$ and host $h \in U$, we define:

$$\mathrm{out}_e(h) \triangleq \sum_{(h,k) \in A} x_{e,h,k} \qquad \text{and} \qquad \mathrm{in}_e(h) \triangleq \sum_{(k,h) \in A} x_{e,k,h}$$

We enforce:

$$\mathrm{in}_e(h) \le 1 \qquad \forall e \in E,\ \forall h \in U \tag{5}$$

$$\mathrm{out}_e(h) \le 1 \qquad \forall e \in E,\ \forall h \in U \tag{6}$$

$$\mathrm{out}_e(h) - \mathrm{in}_e(h) = y_{u,h} - y_{v,h} \qquad \forall e = (u, v) \in E,\ h \in U \tag{7}$$

Eqs. (5)–(6) prevent branching or merging for a single logical edge by limiting each commodity to at most one incoming and one outgoing arc at any host. Eq. (7) enforces flow conservation everywhere except at the selected source and sink: if u is placed on h, the commodity has net +1 at h (one unit must leave); if v is placed on h, it has net -1 (one unit must arrive). Together, these constraints ensure that each circuit edge is realized as a single directed route between host endpoints.

(iii) Path correctness constraints. A routed signal should not traverse a PUC that is actively implementing a circuit operation not on the signal's path in photonic circuit graph, since that would introduce an unintended transformation. Thus, paths may pass through a host node only if it is *unused*, unless the host node is the source or sink for the signal.

Let $\mathrm{use}(h) = \sum_{v:\, h \in C(v)} y_{v,h} \in \{0, 1\}$ indicate whether host h is occupied by some circuit vertex (Eq. (2) makes this binary). Then for all $e = (u, v) \in E$ and $h \in U$:

$$\mathrm{in}_e(h) \le 1 - \mathrm{use}(h) + y_{u,h} + y_{v,h} \quad \forall e \in E,\ h \in U \tag{8}$$

$$\mathrm{out}_e(h) \le 1 - \mathrm{use}(h) + y_{u,h} + y_{v,h} \quad \forall e \in E,\ h \in U \tag{9}$$

If h is unused ($\mathrm{use}(h) = 0$), these constraints reduce to $\mathrm{in}_e(h) \le 1$ and $\mathrm{out}_e(h) \le 1$ and do not further restrict routing. If h is occupied ($\mathrm{use}(h) = 1$) and is not an endpoint for e, the right-hand side becomes 0, forcing $\mathrm{in}_e(h) = \mathrm{out}_e(h) = 0$; i.e., e's route cannot pass through h.

Finally, we require disjoint edges in the photonic circuit graph to use disjoint physical links on the hardware:

$$\sum_{e \in E} x_{e,i,j} \le 1 \qquad \forall \{i, j\} \in A. \tag{10}$$

(iv) Geometry constraints. We now enforce constraints that depend on the physical faces of each PUC.

Same-side inputs. If a labeled PUC implements a multi-input operation (e.g., `unitary`), its inputs must enter from a single physical side. In Figure 7, if a circuit vertex v is placed on host 3, then both inputs must arrive either from the left side ($\{1, 2\}$) or from the right side ($\{4, 5\}$), but not one from each. We encode this by selecting exactly one side group when placing v on h:

$$\sum_{s=1}^{k_h} g_{v,h,s} = y_{v,h} \qquad \forall v \in V,\ \forall h \in C(v) \tag{11}$$

For every incoming logical edge $e = (u, v) \in E$, we require the final hop into h to come from the chosen side group:

$$\sum_{\substack{(p,h) \in A \\ p \notin S_{h,s}}} x_{e,p,h} \le 1 - g_{v,h,s} \quad \begin{array}{l} \forall v \in V,\ e = (u, v) \in E, \\ \forall h \in C(v),\ s \in \{1, \ldots, k_h\} \end{array} \tag{12}$$

If $g_{v,h,s} = 1$, Eq. (12) forces e's incoming arc into h to originate from a neighbor $p \in S_{h,s}$, ensuring all inputs to v enter from the same physical side (outputs remain unrestricted).

No same-side V-turns. Some meshes cannot realize a route that enters and leaves a PUC via two ports on the same physical face (a "V-turn"), *e.g.,* in Figure 7, entering host 3 from 1 and leaving to 2 makes a V-turn on the left side. For all hosts h, logical edge e, side group $S_{h,s}$, and distinct neighbors $p \neq q$ with $p, q \in S_{h,s}$, we forbid same-side V-turns:

$$x_{e,p,h} + x_{e,h,q} \leq 1. \tag{13}$$

Objective. We minimize total signal loss by minimizing the weight of all physical arcs used by all routed edges:

$$\min \sum_{e \in E} \sum_{(i,j) \in A} w_{ij}\, x_{e,i,j}. \tag{14}$$

Eq. (14) with the constraints above yields a complete ILP for mapping a photonic circuit graph onto a physical mesh. While our implementation minimizes the total loss, the same ILP admits other objectives. For example, we can enforce a per-path budget as follows: Let $L_e = \sum_{(i,j) \in A} w_{ij} x_{e,i,j}$ be the loss on logical edge e. Them $L_e \leq B$ enforces a budget. In general, one can plug different objective functions which will allow the compiler to produce configurations with more predictable and uniform performance.

Calibration-aware embedding. In practice, photonic meshes deviate from their ideal circuit models. Waveguides and PUCs introduce non-uniform loss, phase shifters can drift over time, and some elements may become unavailable. Calibration measures these device-specific effects giving a view of the fabric's real-time behavior.

Our compiler can use this calibration information when choosing an embedding. Measured propagation losses can be encoded as edge weights, unavailable or unstable PUCs can be removed from candidate sets, and calibrated PUC behavior can be reflected in finer-grained label sets, such as restricting a PUC to the phase shifts or splitting ratios it can realize reliably. Thus, λ_λ does not perform low-level device calibration itself, but its compiler can consume calibrated measurements to avoid fragile paths and produce configurations better matched to the physical fabric.

4.2 NP-Completeness of Embedding Feasibility

A natural question is whether there is a more efficient algorithm to embed a photonic circuit graph into a mesh. We show that the embedding problem is NP-complete by formalizing the embedding feasibility question as a decision problem called the *Constrained Graph Embedding (CGE)*.

Theorem 4.1. *CGE is NP-complete.*

We defer the formal definition of the CGE problem and the proof to Appendix E.

5 Language Evaluation

In this section, we use λ_λ to program photonic circuits for several key networking applications. We evaluate λ_λ along two axes: conciseness of λ_λ programs and the performance of the compiled circuits on a real photonic switch. We present four case studies: circuit switching, rotor switching, broadcast, and in-network AllReduce.

```
let b_signal = input(0)
in unpack split(b_signal, 0.5) as (s₁, s₂)
in output(2) <- s₁;  output(39) <- s₂;
output(5) <- input(6);
```

Figure 8: λ_λ program for Broadcast.

Each case study showcases how λ_λ can be used to express complex optical operations concisely and efficiently map them onto physical hardware. We evaluate the performance of the compiled programs on the commercially available iPronics [28] programmable silicon-photonic switch, with 72 programmable unitary cells (PUCs) and 40 input/output ports. Omitted programs and mesh configurations can be found in Appendix A.

5.1 Circuit Switching

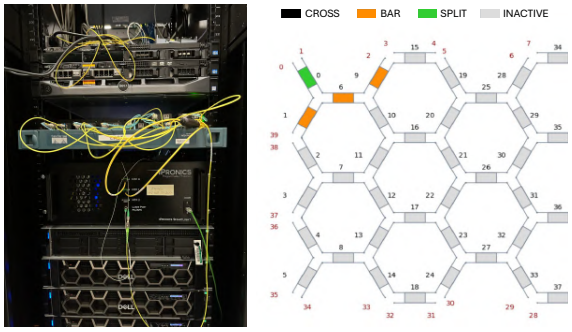
A programming language for photonic circuits should be able to express switching operations, which are fundamental to many networking applications. Such programs can be expressed concisely in λ_λ, leveraging its abstractions for optical components. To implement a circuit switch, we need to redirect optical signals from input ports to output ports. For example, to implement a 4-way switch, we can use the following λ_λ code:

```
output(1) <- input(2);
output(3) <- input(4);
output(5) <- input(6);
output(7) <- input(8);
```

This code snippet routes a signal from port 2 to port 1, and similarly for the other ports, creating a 4-way switch.

Hardware testbed. We compiled this 4-port switch program and evaluated it in hardware. For switching experiments, our hardware testbed (Figure 9a) consists of the iPronics photonic switch and four Dell 7960 rack servers, each with an NVIDIA RTX 6000 Ada GPU (48GB memory) and an Intel E810-XXVDA4 NIC (four 25/10/1 GbE ports). We connect the NIC ports to the iPronics photonic switch by using SFP+ optics and external wavelength conversion. Specifically, we insert FS 10GBASE-SR SFP+ 850 nm transceivers into the NIC and connect them via fibers to FS 1x10G SFP+ to 1x10G SFP+ media converters. Because the iPronics device only accepts 1550 nm light, we insert FS 10GBASE-ER SFP+ 1550 nm transceivers on the iPronics-facing side of the media converters and use fibers to feed the signal into the iPronics switch. However, the iPronics only accepts transverse-electric polarization while commercial transponders typically emit polarization-multiplexed light. Due to this hardware limitation, we had to use lossy converters that resulted in the iPronics only being able to support 10 Gbps of traffic. Thus, we report the normalized bandwidth as a fraction of 10 Gbps in our figures (Figure 10).

In the first set of experiments, we provision two independent bidirectional circuits through the iPronics mesh between two servers. With four Intel NIC ports per server, we can support two full-duplex links across the mesh (one port for Tx and the other for Rx). Thus, we configure two separate interfaces per server and assign one IP address per interface; the two interfaces use different IP subnets. We use λ_λ to compile two bidirectional connections by programming four directed paths between iPronics ports: 5⟶7, 9⟶10, 38⟶37, 39⟶36. To enable connectivity, we connect



(a) Our testbed. (b) Broadcast configuration.

Figure 9: 9a shows our hardware testbed with the iPronics. 9b shows the mesh configuration for the Broadcast operation: input from port 0 is broadcast to output ports 2 and 39.

the Tx of interface 1 of server 1 to port 5 and the Rx of interface 1 of server 2 to port 7, and vice versa for ports 9 and 10. We use the same approach for the other interface on both servers, using the other two paths, thus enabling two bidirectional connections.

To confirm switching functionality, we run 10 Gbps bidirectional `iperf` sessions simultaneously between each pair of interfaces across servers. We plot the `iperf` bandwidth for the bidirectional circuit between the first interfaces on both servers in Figure 10a and between the second interfaces in Figure 10b. Consistent `iperf` bandwidth of 10 Gbps for both circuits confirms that λ_λ can successfully express and compile circuit switching programs for photonic switches.

5.2 Rotor Switching

In addition to basic switching operations, λ_λ can also express *time-varying* switching schedules that reconfigure the mesh. We implement a rotor switch, which repeatedly cycles through a fixed sequence of matchings in order to achieve all-to-all connectivity among a set of ports over a period of time [32]. We express the rotor switch as three different λ_λ programs (shown in Appendix A), each specifying a matching. Executing these programs in a loop cycles through all possible input-output port connections, ensuring predictable all-to-all connectivity over time.

We compile the rotor switch programs and evaluate them on the iPronics switch using the same hardware testbed, now with all four servers. We connect the first interface of all four servers to the switch and execute three λ_λ programs in a fixed cycle, switching programs every 10 seconds (*i.e.,* reconfiguring the mesh between timeslots). We choose a 10-second timeslot to accommodate switch reconfiguration latency and obtain stable `iperf` measurements. In each timeslot, the fabric configures a fixed matching (*e.g.,* $1 \leftrightarrow 4$ and $2 \leftrightarrow 3$, then $2 \leftrightarrow 4$ and $1 \leftrightarrow 3$, and $3 \leftrightarrow 4$ and $1 \leftrightarrow 2$). Over a 30-second period, all pairs of servers will have been connected. To demonstrate feasibility, we run an `iperf` server on server 4 and three `iperf` clients on servers 1-3, each sending at 10 Gbps. Figure 10c plots server 4's measured `iperf` throughput over time. In each timeslot, exactly one client achieves 10 Gbps while the other two clients are disconnected. This cycle repeats in a periodic pattern every 30 seconds, confirming correct rotor-switching operation in hardware.

5.3 Broadcast

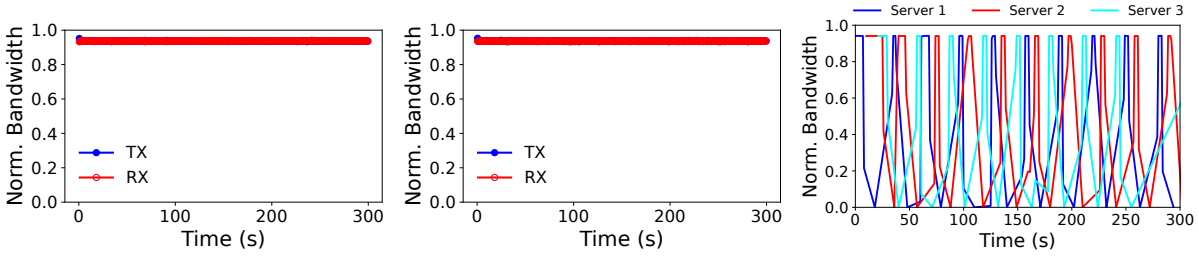
Broadcasting is a fundamental networking primitive in which a single sender simultaneously transmits data to multiple recipients. We implement broadcasting with a single sender and two receivers using the *split* operation as shown in Figure 8. We add an extra path from port 6 to 5 to create a bidirectional link in the mesh, as Ethernet requires full duplex links to recognize the link as up. Figure 9b shows the partial mesh configurations generated from our λ_λ broadcast program. We execute this program in our hardware testbed, connecting three of the servers (server 1 as the sender and servers 2 and 3 as the recipients) to the iPronics. However, off-the-shelf Ethernet hardware only recognizes a link as up if it can detect both a Tx and Rx signal. On server 1, we match its Tx with server 2's Rx, but there is no corresponding Tx to match server 3's Rx, since we split the signal *in the mesh*. As a result, link-layer protocols cannot detect a second link (between server 1 and server 3) because there is no matching Tx on server 1 for server 3's Rx, despite being able to detect power on server 3's Rx.

Due to these limitations with link-layer protocols that only enable a logical link between servers 1 and 2 (despite a physical link also existing between servers 1 and 3), we run an `iperf` session only between servers 1 and 2. We confirm ~ 10 Gbps of bandwidth, and measure Rx power of -16 dBm on server 3. These results show (1) that server 3 received the signal at the physical layer, as it received power much higher than the noise floor of -49 dBm, and (2) that despite splitting the signal, which incurs an additional 3 dB of loss [13], our photonic switch ensures Layer 3 communication between servers at the maximum supported bandwidth.

5.4 In-Network AllReduce

AllReduce is a critical collective communication primitive used in distributed ML workloads [43]. The AllReduce primitive transforms a precondition in which every node has its own data buffer into a postcondition where all nodes share a common data buffer that is the result of an elementwise aggregation operation across nodes; this is the elementwise sum or average in the data- and tensor-parallel sharding strategies used in distributed ML. Inspired by prior work that *manually* designs mesh configurations to realize optical in-network AllReduce [13], we use λ_λ to program mesh configurations to achieve the collective operation. The AllReduce operation takes multiple input signals, computes their average in the analog photonic domain, and distributes the result back to all output ports.

Optical All-Reduce. To implement optical averaging in silicon photonics, we use the same technique as prior work [13]: a unitary transformation known as the Hadamard transform that averages optical power. A 2×2 Hadamard matrix is defined as: $\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}$. To understand how this matrix operates on input signals, consider two light beams $A_1 \cdot e^{i\phi_1}$ and $A_2 \cdot e^{i\phi_2}$ where A_1 and A_2 are the amplitudes and ϕ_1 and ϕ_2 are the phases of the beams. Let $\begin{pmatrix}E_1'\\ E_2'\end{pmatrix} \triangleq \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}A_1e^{i\phi_1}\\ A_2e^{i\phi_2}\end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix}A_1e^{i\phi_1} + A_2e^{i\phi_2}\\ A_1e^{i\phi_1} - A_2e^{i\phi_2}\end{pmatrix}$. The output intensities (proportional to optical power) at the two output ports are given by: $|E_1'|^2 = \frac{1}{2}\left(A_1^2 + A_2^2 + 2A_1A_2\cos(\phi_1 - \phi_2)\right)$ and $|E_2'|^2 = \frac{1}{2}\left(A_1^2 + A_2^2 - 2A_1A_2\cos(\phi_1 - \phi_2)\right)$. For incoherent inputs, the



(a) Interface 1 circuit. (b) Interface 2 circuit. (c) RotorSwitch implementation.

Figure 10: Performance of switching applications on the iPronics photonic switch, using compiled λ_λ programs.

```
let x1 = input(1) in
let x2 = input(2) in
let x3 = input(3) in
let x4 = input(4) in
unpack unitary(H, (x1, x2)) as y1, y2 in
unpack unitary(H, (x3, x4)) as y3, y4 in
unpack unitary(H, (y1, y3)) as z1, z2 in
unpack unitary(H, (y2, y4)) as z3, z4 in
output(5) <- z1;
output(6) <- z2;
output(7) <- z3;
output(8) <- z4;()
```

Figure 11: 4-way AllReduce program in λ_λ. Here H denotes a 2x2 Hadamard matrix.

interference term vanishes and both outputs carry half the total power: $|E_1'|^2 = |E_2'|^2 = (A_1^2 + A_2^2)/2$, giving the average optical power.

Implementation in λ_λ. We implement a 4-way AllReduce operation in λ_λ using a series of Hadamard transforms as shown in Figure 11. The program takes four input signals. It first computes pairwise averages of the inputs using two Hadamard transforms, and then combines the results using two more Hadamard transforms to produce the final averaged outputs.

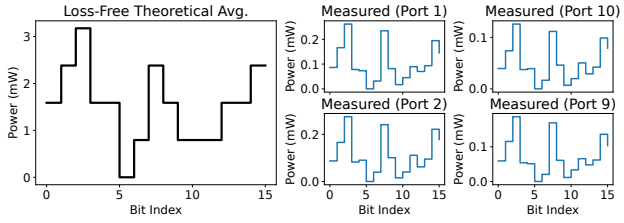


Figure 12: Analog AllReduce in photonic hardware. Left panel shows the theoretical average power given inputs while the right shows the measured power at each output port; the trend closely matches, confirming accurate AllReduce.

Hardware Evaluation. We compile the AllReduce program and evaluate it on the iPronics device. In this experiment, our hardware testbed consists of the iPronics photonic switch and four C-band (1550 nm) tunable lasers as inputs. Following prior work [13, 56], we use a simple intensity-based encoding to quantize floating-point numbers into signed integers and transmit them as optical power levels. In the encoding, 0 corresponds to -49 dBm (off) and 1 corresponds to +5.02 dBm. With four inputs and a binary encoding scheme, there are $2^4 = 16$ possible combinations of inputs. Thus, we generate a random sequence of all possible input combinations and evaluate these by configuring each of the lasers to the appropriate power according to its bit value. We then measure the optical power received at the output ports using built-in photodetectors on the iPronics device and compare it to the expected values from theoretically averaging the input powers in Figure 12. (To do so, we must convert from dBm, a logarithmic value, to mW, a linear value.)

Because of insertion and routing losses, the measured output power is expected to be significantly lower than the ideal output power. Further, since we measure the power at each output port using a different photodetector (*i.e.,* the built-in photodetector for that port), the baseline power measurements across output ports will also differ. Thus, we are interested in ensuring similar *trends* given some baseline calibration, and Figure 12 shows a near-identical trend across output ports that closely matches the expected pattern.

5.5 Program Conciseness

We evaluate λ_λ's usability by measuring the lines of code (LOC) required to implement the case studies. Table 2 summarizes the LOC for each case study, comparing it with the required number of PUC configurations. We can clearly see that specifying circuit behavior in λ_λ is easier and more concise than manually configuring individual PUCs, while still being performant on real hardware.

Case Study	LOC	# PUCs
Broadcast	4	7
Circuit Switching	4	16
Rotor Switching (3 programs)	12	37
In-Network AllReduce	12	30

Table 2: Lines of Code (LOC) in the λ_λ language and Number of PUC configurations for each case study.

6 Compiler Evaluation

We now turn to evaluating the scalability of our compiler. We have three major knobs to turn when evaluating the compiler's performance: the size of the photonic mesh, the complexity of the λ_λ program being compiled, and the geometry of the mesh. To evaluate along these dimensions, we quantify the scaling behavior of the compiler with both *strong scaling* (i.e., compile time as the mesh size increases, keeping the program complexity fixed) and

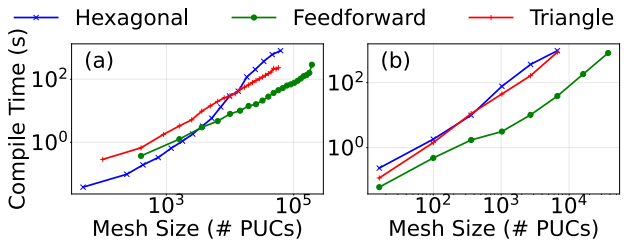


Figure 13: Strong scaling results for different geometries.

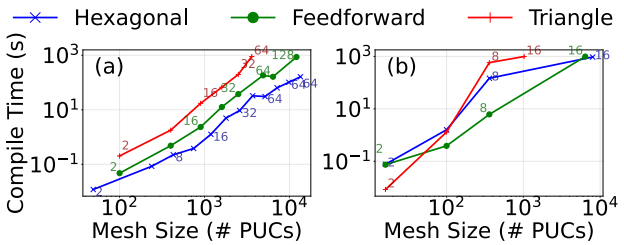


Figure 14: Weak scaling results for different mesh geometries.

weak scaling (i.e., compile time as both the mesh size and program complexity increase) for different mesh geometries.

Mesh Geometries. We focus on three different mesh geometries that have been proposed in prior work [8] as shown in Figure 4. Each geometry, namely *hexagonal*, *triangular*, and *feedforward*, has a different arrangement of PUCs and waveguides, which leads to different connectivity properties, and thus different compilation performance.

Program Complexity. We measure program complexity by the size of the photonic circuit graph, which correlates with λ_λ program size. We scale complexity by enlarging the case-study programs in §5: for switching, by increasing the number of connected input-output port pairs; for AllReduce, by increasing the number of inputs being reduced. We also generate random $n \times n$ unitary matrices and decompose them into sequences of PUCs using standard optical techniques [10], yielding λ_λ programs of increasing size.

Strong Scaling. We evaluate the strong scaling behavior of the compiler by fixing the program and increasing the size of the target mesh. We consider a switch program with 8 input-output pairs and an AllReduce program with 4 inputs. We scale the number of PUCs and waveguides under a 1000*s* timeout. Figure 13(a) isolates the effect of mesh size under a fixed routing workload. The feedforward mesh exhibits the best scaling, supporting over 10^5 PUCs within the time limit, while hexagonal and triangular meshes show similar but slightly weaker scaling. Figure 13(b) shows analogous trends for AllReduce, again with the feedforward mesh performing best. These results show that the compiler scales to large meshes, with performance dependent on mesh geometry.

Weak Scaling. Figure 14 reports weak scaling results, where program size and target mesh size are increased simultaneously. In Figure 14(a), we scale the number of input-output port pairs, capturing scenarios where routing complexity grows with mesh size. Compared to fixed-route experiments, all curves exhibit steeper slopes on log-log axes, indicating that program growth compounds compilation cost beyond mesh size alone. On the feedforward mesh, switch programs scale to 128 input-output pairs within the time limit.

Figure 14(b) shows analogous results for AllReduce, where increasing the number of inputs increases compile time with both program and mesh size. Across all mesh geometries, we scale to AllReduce programs with up to 16 inputs within the time limit. These results demonstrate that the compiler is effective under increasing program complexity and mesh size, with performance dependent on mesh geometry.

Unstructured workload. We evaluate unstructured workloads by converting random $n \times n$ unitary matrices to programs and compiling them. We increase n to scale the program size. As shown in Figure 15, compile time grows with matrix size. The feedforward mesh exhibits the best scaling, supporting up to 9×9 unitaries within the time limit with low variance across instances. The hexagonal and triangular meshes scale similarly but support smaller sizes, up to 7×7 and 5×5 unitary matrices, respectively.

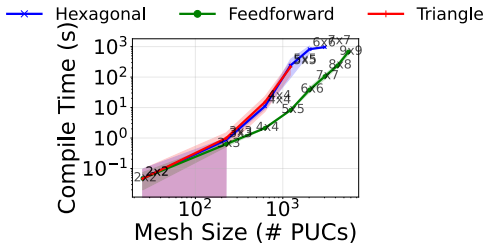


Figure 15: Compile time vs. mesh size for random unitary matrices and different mesh geometries.

7 λ_λ Synthesizer

While λ_λ enables precise control over photonic circuits, programming directly at the level of circuits remains challenging. Users typically begin with a high-level *intent* that does not naturally decompose into optical primitives or respect unitary constraints. Manually translating such intent into a photonic circuit requires computing a unitary completion and optical decomposition. To address this gap, we build a *synthesizer* that automatically generates correct-by-construction λ_λ programs from high-level specifications.

Overview. The synthesizer takes as input a declarative specification describing the desired relationship between input and output ports. From this specification, it (1) infers the *smallest* unitary transformation that realizes the desired functionality (iff one exists), (2) decomposes the inferred unitary into a sequence of PUC configurations using standard optical techniques [10], and (3) emits a corresponding λ_λ program, which can be compiled to a concrete physical mesh by our compiler. This end-to-end pipeline allows users to program photonic hardware at the level of functional behavior, without reasoning about optical primitives.

A key challenge is that users may specify transformations that are not themselves unitary, but such transformations can sometimes be realized by embedding them into a larger unitary matrix using ancillary input and output ports carrying no optical power [40]. Determining whether such an embedding exists, inferring the minimal number of ancillary ports required and the unitary matrix is handled automatically by the synthesizer, which uses standard techniques from linear algebra to find the unitary matrix (see e.g., [26]).

Specification Language. Specifications are written as relations between input and output ports, expressed using linear expressions. On their own, specifications are *not* λ_λ programs. It is the job of the synthesizer to find λ_λ programs that realize a given specification. For example, a simple switching behavior can be specified as `output[i] = input[j]`, while a 2×2 AllReduce operation can be written as `output[1] = (input[1] + input[2])/√2` and `output[2] = (input[1] - input[2])/√2`.

We require users to specify the symbolic relationship between input and output light beams, not the values measured after a particular detection scheme. Photonic measurements are scheme-dependent, and therefore do not provide a universal abstraction for specifying desired behavior. Different measurement schemes yield different observed quantities that lead to different higher-level specifications. Therefore, we design our specification language to describe the desired linear transformation of optical signals, independent of measurement scheme. This avoids the need to commit to a specific optical measurement scheme while still allowing users to express *what* transformation they want rather than *how* it should be realized optically.

Unitary Inference. Given a specification, the synthesizer constructs a system of linear equations capturing the constraints imposed by the desired input-output behavior. If these constraints can be satisfied by a unitary transformation, the synthesizer constructs the *smallest* unitary realizing the specification, adding ancillary ports when necessary. We use standard linear algebra to design an inference procedure that reduces the task to solving systems of linear equations and performing matrix factorizations (Appendix F).

Decomposition and Code Generation. Once a unitary transformation is inferred, we use a technique called *Reck's decomposition* [38], which produces an arrangement of PUCs that corresponds to an $n \times n$ unitary transformation. The resulting sequence of optical primitives is then translated into a λ_λ program using sequencing constructs such as `let` and `unpack`. This automatically generated program is guaranteed to implement the original specification and can be directly compiled to a physical mesh using our compiler.

7.1 Synthesizer Evaluation

We evaluate the synthesizer's scalability by measuring the time required to generate λ_λ programs from high-level specifications, focusing on workloads common in optical networking applications.

Benchmarks. We evaluate the synthesizer on four workloads spanning a variety of applications, ranging from basic switching to realizing arbitrary linea transformation.

- **Switching:** Specified declaratively as `output[i] = input[j]` (§5), this benchmark tests whether the synthesizer can recognize the implied permutation matrix and decompose it into a λ_λ program.
- **AllReduce:** AllReduce computes the average of inputs from multiple sources and broadcasts the result (§5). It is specified as a relation between input and output ports. For the two inputs, the declarative specification will be: `output[1] = (input[1] + input[2])/√2` and `output[2] = (input[1] - input[2])/√2`. Here, the outputs are a normalized sum and difference of the inputs. From this specification, the synthesizer is able to automatically generate a λ_λ program that realizes this transformation. For a 4-way AllReduce, the synthesizer yields a program equivalent to Figure 11.
- **Discrete Fourier Transform (DFT):** The DFT is a core primitive for in-network signal processing (e.g., spectrum analysis and channel estimation). It re-expresses a block of N input signals in terms of their frequency content, where each output port k measures how strongly the input oscillates at frequency k. It is specified directly as the linear relation between input and output ports: $y_k = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x_n e^{-2\pi i k n/N}$, where y_k is the kth output. This evaluates whether the synthesizer can infer the unitary directly from a high-level specification.
- **General linear transformations:** Implementing general linear maps $y = Mx$ enables functionality like in-network matrix multiplication. Since the hardware supports only unitary operations, this benchmark tests the ability to compute unitary embeddings for arbitrary matrices.

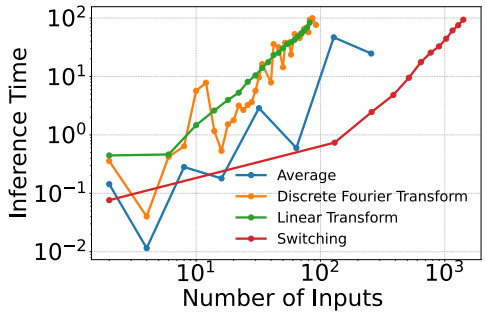


Figure 16: Synthesizer performance across benchmarks.

Results. The scaling results are shown in Figure 16. We observe that the synthesizer scales differently across benchmarks. We set a timeout of 100 seconds for each synthesis run. The switching benchmark, which is the simplest, scales to over 1000 input-output port pairs within the time limit. Our system can synthesize AllReduce programs with up to 256 inputs, and DFT programs with up to 96 inputs. Finally, for random linear transformations, which require unitary embedding and are the most complex, we can synthesize programs with up to 82 inputs within the time limit. This suggests that starting from a specification, our synthesizer can efficiently generate a variety of photonic programs.

8 Related Work

Programmable Network Data Planes. The past decade has seen significant progress in languages for programming network data planes. P4 [7] introduced a target-independent language for specifying packet parsing and match-action processing, enabling protocol-independent forwarding on programmable switches. The Frenetic project [17] and its successors [33] developed higher-level abstractions for SDN programming, culminating in NetKAT [1, 5], which provides a sound and complete equational theory grounded in Kleene algebra with tests. Subsequent work extended these foundations to handle stateful packet processing: Domino [47] introduced packet transactions as an abstraction for line-rate stateful algorithms, SNAP [2] provided network-wide stateful abstractions, and Lucid [48] offered event-driven programming for the data plane. These systems share a common goal with λ_λ: elevating network

devices from fixed-function hardware to programmable substrates. However, they target electrical packet switches with fundamentally different constraints than photonic circuits. Where P4 programs reason about header fields and match-action tables, λ_λ programs reason about optical signals and physical path constraints.

Linear Types and Resource Management. Linear type systems, originating with Girard's linear logic [21], have been used extensively in programming languages research to enforce resource usage [15, 53, 54]. The closest to our work is the use of linear types in quantum computing [6, 18, 19, 41, 49, 57], to enforce no duplication of qubits.

Optical Network Programming. While programmable electrical switches have received significant attention, programming abstractions for optical networks remain nascent. Existing optical circuit switches are configured via low-level vendor APIs or simple path-provisioning interfaces [16, 37, 55]. Recent silicon photonic switches enable topology engineering [14, 28] and in-network analog computation [13], but require manual specification of physical parameters. Edgewise [4] compiles high-level policies over ROADMs using augmented NetKAT. To our knowledge, λ_λ is the first language to provide abstractions for programming silicon photonic switches, with a novel type system and a compiler that automatically maps programs to arbitrary mesh topologies.

9 Conclusion

This paper introduced λ_λ, a programming language and compiler stack for silicon photonic circuit switches that gives programmers the ability to harness programmable silicon photonics as a medium of compute, making programmable in-network compute a reality. λ_λ's key innovations are a linear type system that encodes optical signals as a resource, and a compiler that maps photonic programs onto arbitrary mesh geometries. Together these contributions make it possible to express rich optical behaviors without manual low-level tuning. Our evaluation demonstrates that λ_λ simplifies programming and produces efficient mappings across diverse meshes. By providing the first language and compiler for silicon photonic switches, λ_λ paves the way toward more programmable, verifiable, and productive optical networks. We envision higher level-languages, and more diverse networking applications that use λ_λ as a foundation.

Ethics. This work does not raise any ethical issues.

Acknowledgements

We thank our anonymous shepherd for their feedback. This work benefited from discussions with Eric Ding and Fred Schneider.

This work was supported in part by ACE, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA. This work is partially supported by grants from the by NSF (Awards No. 2444537, 2435852, 2319186 and 2152831AD), and the Office of Naval Research (N00014-23-1-2134). AD is supported by the NSF Graduate Research Fellowship.

References

[1] Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. 2014. NetKAT: semantic foundations for networks. *SIGPLAN Not.* 49, 1 (jan 2014), 113–126. https://doi.org/10.1145/2578855.2535862

[2] Mina Tahmasbi Arashloo, Yaron Koral, Michael Greenberg, Jennifer Rexford, and David Walker. 2016. SNAP: Stateful Network-Wide Abstractions for Packet Processing. In *SIGCOMM*.

[3] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, István Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, Benn Thomsen, and Hugh Williams. 2020. Sirius: A Flat Datacenter Network with Nanosecond Optical Switching. In *SIGCOMM '20: Proceedings of the 2020 Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication, Virtual Event, USA, August 10-14, 2020*, Henning Schulzrinne and Vishal Misra (Eds.). ACM, 782–797. https://doi.org/10.1145/3387514.3406221

[4] Shrutarshi Basu, Nate Foster, Hossein Hojjat, Paparao Palacharla, Christian Skalka, and Xi Wang. 2017. Life on the Edge: Unraveling Policies into Configurations. In *2017 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 178–190. https://doi.org/10.1109/ANCS.2017.31

[5] Ryan Beckett, Michael Greenberg, and David Walker. 2016. Temporal NetKAT. In *PLDI*. 386–401.

[6] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: a high-level quantum language with safe uncomputation and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 286–300. https://doi.org/10.1145/3385412.3386007

[7] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (July 2014), 87–95. https://doi.org/10.1145/2656877.2656890

[8] José Capmany and Daniel Pérez. 2020. *Programmable integrated photonics*. Oxford University Press.

[9] Lukas Chrostowski and Michael Hochberg. 2015. *Silicon Photonics Design: From Devices to Systems*. Cambridge University Press. https://doi.org/10.1017/CBO9781316084168

[10] William R. Clements, Peter C. Humphreys, Benjamin J. Metcalf, W. Steven Kolthammer, and Ian A. Walmsley. 2016. Optimal design for universal multiport interferometers. *Optica* 3, 12 (Dec 2016), 1460–1465. https://doi.org/10.1364/OPTICA.3.001460

[11] Charles Clos. 1953. A study of non-blocking switching networks. *The Bell System Technical Journal* 32, 2 (1953), 406–424. https://doi.org/10.1002/j.1538-7305.1953.tb01433.x

[12] NVIDIA Corporation. [n. d.]. Co-Packaged Silicon Photonics Networking Switches. https://www.nvidia.com/en-us/networking/products/silicon-photonics/. ([n. d.]). Accessed: 2025-04-08.

[13] Arjun Devraj, Bill Owens, Daniel Pérez-López, and Rachee Singh. 2026. In-Network Analog AllReduce for ML with Programmable Integrated Photonics. In *2026 Optical Fiber Communications Conference and Exhibition (OFC)*. 1–3.

[14] Eric Ding and Rachee Singh. 2025. PipSwitch: A Circuit Switch Using Programmable Integrated Photonics, In Optical Fiber Communication Conference (OFC) 2025. *Optical Fiber Communication Conference (OFC) 2025*, W2A.41. https://doi.org/10.1364/OFC.2025.W2A.41

[15] Manuel Fahndrich and Robert DeLine. 2002. Adoption and focus: practical linear types for imperative programming. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI '02)*. Association for Computing Machinery, New York, NY, USA, 13–24. https://doi.org/10.1145/512529.512532

[16] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiahu Fainman, George Papen, and Amin Vahdat. 2010. Helios: A Hybrid Electrical/Optical Switch Architecture for Modular Data Centers. *SIGCOMM Comput. Commun. Rev.* 40, 4 (aug 2010), 339–350. https://doi.org/10.1145/1851275.1851223

[17] Nate Foster, Rob Harrison, Michael J. Freedman, Christopher Monsanto, Jennifer Rexford, Alec Story, and David Walker. 2011. Frenetic: A Network Programming Language. In *ICFP*. 279–291.

[18] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2022. Proto-Quipper with dynamic lifting. (2022). arXiv:cs.PL/2204.13041 https://arxiv.org/abs/2204.13041

[19] Peng Fu, Kohei Kishida, and Peter Selinger. 2022. Linear Dependent Type Theory for Quantum Programming Languages. *Logical Methods in Computer Science* Volume 18, Issue 3 (Sept. 2022). https://doi.org/10.46298/lmcs-18(3:28)2022

[20] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireeja Ranade, Pierre-Alexandre Blanche, Houman Rastegarfar, Madeleine Glick, and Daniel Kilper. 2016. ProjecToR: Agile Reconfigurable Data Center Interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16)*. Association for Computing Machinery, New York, NY, USA, 216–229. https://doi.org/10.1145/2934872.2934911

[21] Jean-Yves Girard. 1987. Linear logic. *Theoretical computer science* 50, 1 (1987), 1–101.

[22] Albert Greenberg, James R. Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A. Maltz, Parveen Patel, and Sudipta

Sengupta. 2009. VL2: A Scalable and Flexible Data Center Network. In *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication (SIGCOMM '09)*. Association for Computing Machinery, New York, NY, USA, 51–62. https://doi.org/10.1145/1592568.1592576

[23] Navid Hamedazimi, Zafar Qazi, Himanshu Gupta, Vyas Sekar, Samir R. Das, Jon P. Longtin, Himanshu Shah, and Ashish Tanwer. 2014. FireFly: A Reconfigurable Wireless Data Center Fabric Using Free-Space Optics. In *Proceedings of the 2014 ACM Conference on SIGCOMM (SIGCOMM '14)*. Association for Computing Machinery, New York, NY, USA, 319–330. https://doi.org/10.1145/2619239.2626328

[24] Nicholas C. Harris et al. 2018. Linear programmable nanophotonic processors. *Optica* 5, 12 (2018), 1623–1631. https://doi.org/10.1364/OPTICA.5.001623

[25] Nicholas C Harris, Darius Bunandar, Ajay Joshi, Ayon Basumallik, and Robert Turner. 2022. Passage: A wafer-scale programmable photonic communication substrate. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 1–26.

[26] Roger A. Horn and Charles R. Johnson. 1985. *Matrix Analysis*. Cambridge University Press.

[27] Kazuhiro Ikeda, Keijiro Suzuki, Ryotaro Konoike, Shu Namiki, and Hitoshi Kawashima. 2020. Large-scale silicon photonics switch based on 45-nm CMOS technology. *Optics Communications* 466 (2020), 125677.

[28] iPronics. 2026. iPronics ONE Silicon Photonics Optical Switch. https://ipronics.com/product/. (2026).

[29] Warren Jin, Avi Feshali, Mario Paniccia, and John E. Bowers. 2021. Seamless multi-reticle photonics. *Opt. Lett.* 46, 12 (Jun 2021), 2984–2987. https://doi.org/10.1364/OL.427289

[30] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. (2023). arXiv:cs.AR/2304.01433 https://arxiv.org/abs/2304.01433

[31] Vaibhav Mehta. 2026. LambdaLambda Compiler. (June 2026). https://github.com/vaibhavmehta1/LambdaLambda/

[32] William M. Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C. Snoeren, and George Porter. 2017. RotorNet: A Scalable, Low-Complexity, Optical Datacenter Network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '17)*. Association for Computing Machinery, New York, NY, USA, 267–280. https://doi.org/10.1145/3098822.3098838

[33] Christopher Monsanto, Joshua Reich, Nate Foster, Jennifer Rexford, and David Walker. 2013. Composing Software Defined Networks. In *NSDI*.

[34] Jeremie Eliahou Ontiveros, Dylan Patel, and Wei Zhou. 2025. xAI's Colossus 2 - First Gigawatt Datacenter In The World, Unique RL Methodology, Capital Raise. SemiAnalysis. (Sept. 2025). https://newsletter.semianalysis.com/p/xais-colossus-2-first-gigawatt-datacenter Accessed: 2026-01-23.

[35] Daniel Pérez, Ivana Gasulla, José Capmany, and Richard A. Soref. 2016. Reconfigurable lattice mesh designs for programmable photonic processors. *Opt. Express* 24, 11 (May 2016), 12093–12106. https://doi.org/10.1364/OE.24.012093

[36] Benjamin C. Pierce. 2004. *Advanced Topics in Types and Programming Languages*. The MIT Press.

[37] George Porter, Richard Strong, Nathan Farrington, Alex Forencich, Pang Chen-Sun, Tajana Rosing, Yeshaiahu Fainman, George Papen, and Amin Vahdat. 2013. Integrating Microsecond Circuit Switching into the Data Center. *SIGCOMM Comput. Commun. Rev.* 43, 4 (aug 2013), 447–458. https://doi.org/10.1145/2534169.2486007

[38] Michael Reck, Anton Zeilinger, Herbert J. Bernstein, and Philip Bertani. 1994. Experimental realization of any discrete unitary operator. *Phys. Rev. Lett.* 73 (Jul 1994), 58–61. Issue 1. https://doi.org/10.1103/PhysRevLett.73.58

[39] Graham T. Reed and Andrew P. Knights. 2004. *Silicon Photonics: An Introduction*. John Wiley & Sons. https://doi.org/10.1002/0470014180

[40] JJ Schäffer. 1955. On unitary dilations of contractions. In *Proc. Amer. Math. Soc*, Vol. 6. 322.

[41] PETER SELINGER and BENOIT VALIRON. 2006. A lambda calculus for quantum computation with classical control. *Mathematical Structures in Computer Science* 16, 3 (2006), 527–552. https://doi.org/10.1017/S0960129506005238

[42] Yichen Shen et al. 2017. Deep learning with coherent nanophotonic circuits. *Nature Photonics* 11 (2017), 441–446. https://doi.org/10.1038/nphoton.2017.93

[43] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *CoRR* abs/1909.08053 (2019). arXiv:1909.08053 http://arxiv.org/abs/1909.08053

[44] Vishal Shrivastav, Asaf Valadarsky, Hitesh Ballani, Paolo Costa, Ki Suh Lee, Han Wang, Rachit Agarwal, and Hakim Weatherspoon. 2019. Shoal: A Network Architecture for Disaggregated Racks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, 255–270. https://www.usenix.org/conference/nsdi19/presentation/shrivastav

[45] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. 2015. Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google's Datacenter Network. In *Sigcomm '15*.

[46] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. 2015. Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google's Datacenter Network. In *Sigcomm '15*.

[47] Anirudh Sivaraman, Alvin Cheung, Mihai Budiu, Changhoon Kim, Mohammad Alizadeh, Hari Balakrishnan, George Varghese, Nick McKeown, and Steve Licking. 2016. Packet Transactions: High-Level Programming for Line-Rate Switches. In *SIGCOMM*. 15–28.

[48] John Sonchack, Devon Loehr, Jennifer Rexford, and David Walker. 2021. Lucid: A Language for Control in the Data Plane. In *SIGCOMM*.

[49] Sam Staton. 2015. Algebraic Effects, Linearity, and Quantum Programming Languages. *SIGPLAN Not.* 50, 1 (Jan. 2015), 395–406. https://doi.org/10.1145/2775051.2676999

[50] Abhishek Vijaya Kumar, Arjun Devraj, Darius Bunandar, and Rachee Singh. 2024. A case for server-scale photonic connectivity. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks (HotNets '24)*. Association for Computing Machinery, New York, NY, USA, 290–299. https://doi.org/10.1145/3696348.3696856

[51] Abhishek Vijaya Kumar, Eric Ding, Arjun Devraj, Darius Bunandar, and Rachee Singh. 2026. Reconfigurable Torus Fabrics for Multi-tenant ML. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1547–1565. https://doi.org/10.1145/3779212.3790238

[52] Jens Vygen. 1995. NP-completeness of some edge-disjoint paths problems. *Discrete Applied Mathematics* 61, 1 (1995), 83–90. https://doi.org/10.1016/0166-218X(93)E0177-Z

[53] Philip Wadler. 1990. Linear types can change the world!. In *Programming concepts and methods*, Vol. 3. North-Holland, Amsterdam, 5.

[54] David Walker and Kevin Watkins. 2001. On regions and linear types. *ACM Sigplan Notices* 36, 10 (2001), 181–192.

[55] Guohui Wang, David G. Andersen, Michael Kaminsky, Konstantina Papagiannaki, T.S. Eugene Ng, Michael Kozuch, and Michael Ryan. 2010. C-Through: Part-Time Optics in Data Centers. *SIGCOMM Comput. Commun. Rev.* 40, 4 (aug 2010), 327–338. https://doi.org/10.1145/1851275.1851222

[56] Yuepeng Wu, Cen Wang, Hongxiang Guo, Yuta Wakayama, Qiuyan Li, Noboru Yoshikane, Takehiro Tsuritani, and Jian Wu. 2025. Optical AllReduce: Ready to Eliminate Intra-DC In-cast Vulnerability for Deep Learning Training. In *Optical Fiber Communication Conference*. Optica Publishing Group, W4D–5.

[57] Charles Yuan and Michael Carbin. 2022. Tower: data structures in Quantum superposition. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 259–288.

[58] Shizhen Zhao, Rui Wang, Junlan Zhou, Joon Ong, Jeffrey C. Mogul, and Amin Vahdat. 2019. Minimal Rewiring: Efficient Live Expansion for Clos Data Center Networks. In *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, Jay R. Lorch and Minlan Yu (Eds.). USENIX Association, 221–234. https://www.usenix.org/conference/nsdi19/presentation/zhao

[59] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. 2024. Resiliency at Scale: Managing Google's TPUv4 Machine Learning Supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 761–774. https://www.usenix.org/conference/nsdi24/presentation/zu

Appendices are supporting material that has not been peer-reviewed

A Additional Programs from §5

This section collects additional material from §5.

RotorSwitch Figure 17 describes the 3 programs required to implement RotorNet. We cycle through three different port matching configurations.

(a) Round 1

```
output(1) <- input(2);
output(3) <- input(4);
output(5) <- input(6);
```

(b) Round 2

```
output(1) <- input(6);
output(3) <- input(2);
output(5) <- input(4);
```

(c) Round 3

```
output(1) <- input(4);
output(3) <- input(6);
output(5) <- input(2);
```

Figure 17: Three different switch programs implementing RotorNet.

Mesh Configurations The mesh configuration for the 4 way switching is in Figure 19, and the mesh configuration for the all-reduce is in Figure 18.

B Operational Semantics of λ_λ

We define the small-step semantics as a relation $(e, \sigma) \to_\iota (e', \sigma')$ where: e is an expression, $\iota : \text{Port} \to \text{Opt}$ is a fixed *input environment*, and $\sigma : \text{Port} \rightharpoonup \text{Opt}$ is a partial *output store*. The environment ι is read-only; only σ changes. We treat `input(p)` as an environment read; it reduces in one step to the optical value $\iota(p)$. We define $e[v/x]$ to be standard capture avoiding substitution.

Runtime optical values. The source language does not contain literal optical values. For the operational semantics and compiler-correctness proof, however, we use a runtime value grammar that extends source values with semantic optical values $a \in \mathcal{V}_{\text{Opt}}$:

$$v ::= r \mid p \mid U \mid () \mid a \mid (v_1, v_2).$$

Source programs cannot mention a; such values arise only from the input environment and from semantic optical operations during evaluation.

We use operations in the mathsf font (phase, amp, etc) as notation for *semantic* operations on optical signals. phase shifts the phase of an optical value, and amp(r, e) scales the amplitude by r.

Reduction Rules We present the small-step operational semantics in Figure 20.

We now show how our rules can be used to evaluate a program. Consider the program:

```
let x = input(1) in output(2) <- phase(π, x); ()
```

Evaluation steps:

$$\begin{aligned}
&(\texttt{let } x = \texttt{input}(1) \texttt{ in } \texttt{output}(2) \leftarrow \mathsf{phase}(\pi, x);\ (),\ \sigma)\\
&\to_\iota\ (\texttt{let } x = \iota(1) \texttt{ in } \texttt{output}(2) \leftarrow \mathsf{phase}(\pi, x);\ (),\ \sigma)\\
&\to_\iota\ (\texttt{output}(2) \leftarrow \mathsf{phase}(\pi, \iota(1));\ (),\ \sigma)\\
&\to_\iota\ (\texttt{output}(2) \leftarrow \mathsf{phase}(\pi, \iota(1));\ (),\ \sigma)\\
&\to_\iota\ ((),\ \sigma[2 \mapsto \mathsf{phase}(\pi, \iota(1))]).
\end{aligned}$$

Thus, this program returns a *unit* value, but it also assigns phase shifted signal from `input(1)` to port number 2.

C Static Semantics (Type System)

To guarantee physical realizability of photonic circuits, each variable and each port number must be used exactly once. Thus, we extend the linear type system with linear contexts tracking variables and ports. We have 2 contexts:

(1) Linear Variable Context (Δ): all variables used exactly once.
(2) Port Context (Π): port numbers used exactly once.

Formally:

$$\Delta ::= \epsilon \mid \Delta, x : \tau$$
$$\Pi ::= \epsilon \mid \Pi, p : \text{Port}$$

Our typing judgment becomes: $\Delta; \Pi \vdash e : \tau$ Every variable in Δ and every port in Π is used exactly once. The axiom rules use exact singleton contexts for variables and ports; unused linear resources cannot be added by weakening. The compound rules combine premise contexts by disjoint union. The T-OptVal rule is a runtime rule for semantic optical values introduced above; it is not source syntax. Figure 21 present all the typing rules. We formalize the typing relation for this linear optical calculus using a judgment of the form $\Delta; \Pi \vdash e : \tau$. Intuitively, this judgment states that under a linear variable context Δ and a hardware-mapped port context Π, the expression e evaluates to a value of type τ. Following the standard linear typing discipline, the contexts Δ and Π are treated as resources that must be consumed exactly once. This is reflected in the rules (T-Pair) and (T-Let-Lin), where the available resources are partitioned between sub-expressions, and combined only via a disjoint union. If a variable or port appears in the context but is not utilized in the derivation of the term, the term is considered ill-typed. The system distinguishes between pure, unrestricted constants—such as real numbers $\mathbb{R}$ and unitary transformations Unitary and linear resources like Opt (optical signals) and Port. The (T-Input) rule consumes a specific physical port p to produce a first-class optical signal, while (T-Output) acts as a resource sink, recombining a signal with a port to yield a result. Operations like (T-Split) and (T-Unitary) demonstrate the transformation of these signals, where (T-Split) consumes a single signal to produce a pair, and (T-Unitary) orchestrates a joint transformation on two signals. By enforcing this strict context management, the type system provides a static guarantee that optical signals are neither duplicated nor discarded.

Next, we prove a syntactic lemma that says variables are in the context are used exactly once. This will be useful later in proving the main guarantee of our type system.

Lemma C.1.

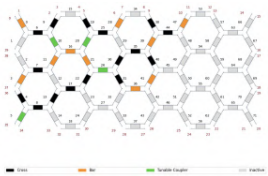


Figure 18: Mesh configuration for 4-way all reduce

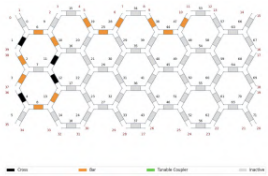


Figure 19: Mesh configuration for 4 way switching program

$$\frac{(e_1,\sigma)\to_\iota(e_1',\sigma')}{(\texttt{let } x=e_1\texttt{ in } e_2,\sigma)\to_\iota(\texttt{let } x=e_1'\texttt{ in } e_2,\sigma')}\ \text{E-Let1} \qquad \frac{}{(\texttt{let } x=v\texttt{ in } e_2,\sigma)\to_\iota(e_2[v/x],\sigma)}\ \text{E-Let2} \qquad \frac{}{(\texttt{input}(p),\sigma)\to_\iota(\iota(p),\sigma)}\ \text{E-Input}$$

$$\frac{(e_1,\sigma)\to_\iota(e_1',\sigma')}{((e_1,e_2),\sigma)\to_\iota((e_1',e_2),\sigma')}\ \text{E-Pair1} \qquad \frac{(e_2,\sigma)\to_\iota(e_2',\sigma')}{((v,e_2),\sigma)\to_\iota((v,e_2'),\sigma')}\ \text{E-Pair2}$$

$$\frac{(e_0,\sigma)\to_\iota(e_0',\sigma')}{(\texttt{unpack } e_0\texttt{as } (x_1,x_2)\texttt{in } e,\sigma)\to_\iota(\texttt{unpack } e_0'\texttt{as } (x_1,x_2)\texttt{in } e,\sigma')}\ \text{E-Unpack1}$$

$$\frac{}{(\texttt{unpack } (v_1,v_2)\texttt{as } (x_1,x_2)\texttt{in } e,\sigma)\to_\iota(e[x_1\mapsto v_1,x_2\mapsto v_2],\sigma)}\ \text{E-Unpack2} \qquad \frac{(e,\sigma)\to_\iota(e',\sigma')}{(\texttt{phase}(\theta,e),\sigma)\to_\iota(\texttt{phase}(\theta,e'),\sigma')}\ \text{E-Phase1}$$

$$\frac{}{(\texttt{phase}(\theta,v),\sigma)\to_\iota(\texttt{phase}(\theta,v),\sigma)}\ \text{E-Phase2} \qquad \frac{(e,\sigma)\to_\iota(e',\sigma')}{(\texttt{split}(r,e),\sigma)\to_\iota(\texttt{split}(r,e'),\sigma')}\ \text{E-Split1}$$

$$\frac{}{(\texttt{split}(r,v),\sigma)\to_\iota((\texttt{amp}(v,r),\texttt{amp}(v,1-r)),\sigma)}\ \text{E-Split2} \qquad \frac{(e_1,\sigma)\to_\iota(e_1',\sigma')}{(\texttt{unitary}(U,(e_1,e_2)),\sigma)\to_\iota(\texttt{unitary}(U,(e_1',e_2)),\sigma')}\ \text{E-Unitary1}$$

$$\frac{(e_2,\sigma)\to_\iota(e_2',\sigma')}{(\texttt{unitary}(U,(v_1,e_2)),\sigma)\to_\iota(\texttt{unitary}(U,(v_1,e_2')),\sigma')}\ \text{E-Unitary2}$$

$$\frac{}{(\texttt{unitary}(U,(v_1,v_2)),\sigma)\to_\iota(((U\cdot\begin{bmatrix}v_1\\v_2\end{bmatrix})[0],\ (U\cdot\begin{bmatrix}v_1\\v_2\end{bmatrix})[1]),\sigma)}\ \text{E-Unitary3}$$

$$\frac{(e_1,\sigma)\to_\iota(e_1',\sigma')}{(\texttt{output}(p)\leftarrow e_1;\ e_2,\sigma)\to_\iota(\texttt{output}(p)\leftarrow e_1';\ e_2,\sigma')}\ \text{E-Out1} \qquad \frac{}{(\texttt{output}(p)\leftarrow v;\ e_2,\sigma)\to_\iota(e_2,\sigma[p\mapsto v])}\ \text{E-Out2}$$

Figure 20: Small-step operational semantics for λ_λ.

Define the occurrence count *of a variable x in a term e by*

$$occ(x,e)\ =\ \begin{cases}1, & e=x,\\ occ(x,e_0), & e=\texttt{phase}(\theta,e_0),\\ occ(x,e_0), & e=\texttt{split}(r,e_0),\\ occ(x,e_1)+occ(x,e_2), & e=\texttt{unitary}(U,(e_1,e_2)),\\ occ(x,e_1)+occ(x,e_2), & e=(e_1,e_2),\\ occ(x,e_1)+occ(x,e_2), & e=\texttt{let } y=e_1\texttt{ in } e_2,\\ occ(x,e_0)+occ(x,e_2), & e=\texttt{unpack } e_0\texttt{ as } (y_1,y_2)\texttt{ in } e_2,\\ occ(x,e_1)+occ(x,e_2), & e=\texttt{output}(p)\leftarrow e_1;\ e_2,\\ 0, & \textit{otherwise.}\end{cases}$$

Then:

If $\Delta;\Pi\vdash e:\tau$ and $x:\tau_x\in\Delta$, then $occ(x,e)=1$.

Proof. By induction on the typing derivation $\Delta;\Pi\vdash e:\tau$. We only show the non-trivial rules; the others are immediate.

Case (T-Var).

$$x:\tau;\emptyset\ \vdash\ x:\tau\ \text{T-Var}$$

Here the exact variable context is $x:\tau$, and $e=x$, so by definition $occ(x,x)=1$.

Case (T-Phase).

$$\frac{\Delta;\Pi\vdash e_0:\text{Opt}\qquad \emptyset;\emptyset\vdash\theta:\mathbb{R}}{\Delta;\Pi\vdash\texttt{phase}(\theta,e_0):\text{Opt}}\ \text{T-Phase}$$

$$\text{T-Var}\ \frac{}{x : \tau; \emptyset \vdash x : \tau} \qquad \text{T-Port}\ \frac{}{\emptyset; p : \mathsf{Port} \vdash p : \mathsf{Port}} \qquad \text{T-Const}\ \frac{}{\emptyset; \emptyset \vdash r : \mathbb{R}} \qquad \text{T-Unit}\ \frac{}{\emptyset; \emptyset \vdash () : \mathsf{Unit}} \qquad \text{T-U-Const}\ \frac{}{\emptyset; \emptyset \vdash U : \mathsf{Unitary}}$$

$$\text{T-OptVal}\ \frac{a \in \mathcal{V}_{\mathsf{Opt}}}{\emptyset; \emptyset \vdash a : \mathsf{Opt}} \qquad \text{T-Let-Lin}\ \frac{\Delta; \Pi \vdash e_1 : \tau_1 \qquad \Delta', x : \tau_1; \Pi' \vdash e_2 : \tau_2}{\Delta, \Delta'; \Pi, \Pi' \vdash \mathtt{let}\ x = e_1 \mathtt{in}\ e_2 : \tau_2} \qquad \text{T-Input}\ \frac{}{\emptyset; p : \mathsf{Port} \vdash \mathtt{input}(p) : \mathsf{Opt}}$$

$$\text{T-Output}\ \frac{\Delta_1; \Pi_1 \vdash e_1 : \mathsf{Opt} \qquad \Delta_2; \Pi_2 \vdash e_2 : \tau}{\Delta_1, \Delta_2; \Pi_1, \Pi_2, p : \mathsf{Port} \vdash \mathtt{output}(p) \mathtt{<-} e_1; e_2 : \tau} \qquad \text{T-Phase}\ \frac{\Delta; \Pi \vdash e : \mathsf{Opt} \qquad \emptyset; \emptyset \vdash \theta : \mathbb{R}}{\Delta; \Pi \vdash \mathtt{phase}(\theta, e) : \mathsf{Opt}}$$

$$\text{T-Split}\ \frac{\Delta; \Pi \vdash e : \mathsf{Opt} \qquad \emptyset; \emptyset \vdash r : \mathbb{R}}{\Delta; \Pi \vdash \mathtt{split}(r, e) : (\mathsf{Opt} * \mathsf{Opt})}$$

$$\text{T-Unitary}\ \frac{\emptyset; \emptyset \vdash U : \mathsf{Unitary} \qquad \Delta_1; \Pi_1 \vdash e_1 : \mathsf{Opt} \qquad \Delta_2; \Pi_2 \vdash e_2 : \mathsf{Opt}}{\Delta_1, \Delta_2; \Pi_1, \Pi_2 \vdash \mathtt{unitary}(U, (e_1, e_2)) : (\mathsf{Opt} * \mathsf{Opt})} \qquad \text{T-Pair}\ \frac{\Delta_1; \Pi_1 \vdash e_1 : \tau_1 \qquad \Delta_2; \Pi_2 \vdash e_2 : \tau_2}{\Delta_1, \Delta_2; \Pi_1, \Pi_2 \vdash (e_1, e_2) : (\tau_1 * \tau_2)}$$

$$\text{T-Unpack}\ \frac{\Delta; \Pi \vdash e : (\tau_1 * \tau_2) \qquad \Delta', x_1 : \tau_1, x_2 : \tau_2; \Pi' \vdash e' : \tau}{\Delta, \Delta'; \Pi, \Pi' \vdash \mathsf{unpack}\ e\ \mathsf{as}\ (x_1, x_2) \mathtt{in}\ e' : \tau}$$

Figure 21: Typing rules for λ_λ

Since $x \in \Delta$, by the induction hypothesis $occ(x, e_0) = 1$. Hence

$$occ(x, \mathtt{phase}(\theta, e_0)) = occ(x, e_0) = 1.$$

Case (T-Split).

$$\frac{\Delta; \Pi \vdash e_0 : \mathsf{Opt} \qquad \emptyset; \emptyset \vdash r : \mathbb{R}}{\Delta; \Pi \vdash \mathtt{split}(r, e_0) : (\mathsf{Opt} * \mathsf{Opt})}\ \text{T-Split}$$

Again $x \in \Delta$ implies by IH $occ(x, e_0) = 1$, and so $occ(x, \mathtt{split}(r, e_0)) = 1$.

Case (T-Unitary).

$$\frac{\begin{array}{c}\emptyset; \emptyset \vdash U : Unitary \\ \Delta_1; \Pi_1 \vdash e_1 : \mathsf{Opt} \qquad \Delta_2; \Pi_2 \vdash e_2 : \mathsf{Opt}\end{array}}{\Delta_1, \Delta_2; \Pi_1, \Pi_2 \vdash \mathtt{unitary}(U, (e_1, e_2)) : (\mathsf{Opt} * \mathsf{Opt})}\ \text{T-Unitary}$$

Since U uses no variables, x lies in exactly one of Δ_1 or Δ_2. By IH on e_1 or e_2, we obtain $occ(x, e_1) = 1$ and $occ(x, e_2) = 0$, or vice versa. Therefore $occ(x, \mathtt{unitary}(U, (e_1, e_2))) = 1$.

Case (T-Out).

$$\frac{\Delta_1; \Pi_1 \vdash e_1 : \mathsf{Opt} \qquad \Delta_2; \Pi_2 \vdash e_2 : \tau}{\Delta_1, \Delta_2; \Pi_1, \Pi_2, p : \mathsf{Port} \vdash \mathtt{output}(p) \leftarrow e_1;\ e_2 : \tau}\ \text{T-Out}$$

Here $\Delta = \Delta_1, \Delta_2$. If $x \in \Delta_1$, IH gives $occ(x, e_1) = 1$ and $occ(x, e_2) = 0$. If $x \in \Delta_2$, then $occ(x, e_1) = 0$ and IH gives $occ(x, e_2) = 1$. In either case, $occ(x, \mathtt{output}(p) \leftarrow e_1;\ e_2) = 1$.

Case (T-Pair).

$$\frac{\Delta_1; \Pi_1 \vdash e_1 : \tau_1 \qquad \Delta_2; \Pi_2 \vdash e_2 : \tau_2}{\Delta_1, \Delta_2; \Pi_1, \Pi_2 \vdash (e_1, e_2) : (\tau_1 * \tau_2)}\ \text{T-Pair}$$

Here $\Delta = \Delta_1, \Delta_2$. Because linear contexts are disjoint, x lies in exactly one of Δ_1 or Δ_2. If $x \in \Delta_1$, IH on e_1 gives $occ(x, e_1) = 1$, and $x \notin \Delta_2$ implies $occ(x, e_2) = 0$, so $occ(x, (e_1, e_2)) = 1 + 0 = 1$. The other subcase is symmetric.

Case (T-LetLin).

$$\frac{\Delta_1; \Pi_1 \vdash e_1 : \tau_1 \qquad \Delta_2,\ y : \tau_1; \Pi_2 \vdash e_2 : \tau}{\Delta_1, \Delta_2; \Pi_1, \Pi_2 \vdash \mathtt{let}\ y = e_1\ \mathtt{in}\ e_2 : \tau}\ \text{T-LetLin}$$

We must show $occ(x, \mathtt{let}\ y = e_1\ \mathtt{in}\ e_2) = 1$, given $x \in \Delta_1, \Delta_2$. Two subcases:

(1) $\mathbf{y = x}$. Then the premise $\Delta_2, x : \tau_1; \Pi_2 \vdash e_2 : \tau$ binds the very x we count. By the induction hypothesis on e_2, since $x \in \Delta_2, x : \tau_1$, we have $occ(x, e_2) = 1$. Meanwhile $x \notin \Delta_1$, so by IH $occ(x, e_1) = 0$. Hence

$$occ(x, \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2) = occ(x, e_1) + occ(x, e_2) = 1.$$

(2) $\mathbf{y \neq x}$. Then $x \in \Delta_1$. By IH on $\Delta_1; \Pi_1 \vdash e_1 : \tau_1$, we get $occ(x, e_1) = 1$. And $x \notin \Delta_2$ implies by IH $occ(x, e_2) = 0$. Thus

$$occ(x, \mathtt{let}\ y = e_1\ \mathtt{in}\ e_2) = 1 + 0 = 1.$$

Case (T-UnpackLin).

$$\frac{\begin{array}{c}\Delta_0; \Pi_0 \vdash e_0 : (\tau_1, \tau_2) \\ \Delta_2,\ y_1 : \tau_1, y_2 : \tau_2; \Pi_2 \vdash e_2 : \tau\end{array}}{\Delta_0, \Delta_2; \Pi_0, \Pi_2 \vdash \mathsf{unpack}\ e_0\ \mathsf{as}\ (y_1, y_2, e_2)\ \mathtt{in}\ : \tau}\ \text{T-UnpackLin}$$

We must show $occ(x, \mathsf{unpack}\ e_0\ \mathsf{as}\ (y_1, y_2)\ \mathtt{in}\ e_2) = 1$. Again two subcases:

(1) $\mathbf{x = y_i}$ for $i \in \{1, 2\}$. Then by IH on $\Delta_2, y_1 : \tau_1, y_2 : \tau_2 \vdash e_2 : \tau$, we have $occ(y_i, e_2) = 1$. Meanwhile,$x \notin \Delta_0$, so $occ(x, e_0) = 0$. Hence

$$occ(x, \mathsf{unpack}\ e_0\ \mathsf{as}\ (y_1, y_2)\ \mathtt{in}\ e_2) = occ(x, e_0) + occ(x, e_2) = 1.$$

(2) $\mathbf{x \neq y_1, y_2}$. Then $x \in \Delta_0$. By IH on $\Delta_0; \Pi_0 \vdash e_0 : (\tau_1, \tau_2)$, we have $occ(x, e_0) = 1$. Or $x \in \Delta_2$ gives $occ(x, e_2) = 1$. But because of disjoint union, both cannot be true. Thus

$$occ(x, \mathsf{unpack}\ e_0\ \mathsf{as}\ (y_1, y_2, e_2)\ \mathtt{in}\ ) = 1 + 0 = 1.$$

□

D Compiling to Intermediate Representation

We now show how to compile *well-typed* λ_λ programs to an intermediate representation. We will then show that this compilation process is produces well-formed graphs for well-typed programs, and then we will show that the procedure is correct, *i.e.,* a compiled program produces the same optical values as the original λ_λ program. We first define a photonic circuit graph. We compile expressions to directed graphs.

Definition D.1. A photonic circuit graph is a tuple

$$G = (V, E, \ell, \Sigma, O, t, r)$$

where:

- V is a finite set of nodes and $E \subseteq V \times V$ is a set of directed edges.
- $\ell : V \to \{\texttt{port}(p), \texttt{output}(p), \texttt{phase}(\theta), \texttt{splitter}(r), U \in \text{Unitary}, \bot\}$ labels each node with an operation or $\bot$ (placeholder).
- $\Sigma : V \to \text{Vars} \cup \{\bot\}$ assigns nodes to variables ($\bot$ for non-variables).
- $O \subseteq V$ is the set of designated *expression outputs*.
- $t : O \to \{1, \ldots, |O|\}$ is a bijection giving a canonical left-to-right ordering of outputs.
- $r : V \rightharpoonup \mathbb{N}$ records the ordering of outputs of splitters and unitaries (values in $\{1, 2\}$) and of placeholder nodes (values in $\{1, \ldots, k\}$).

Compilation is defined relative to a fixed input environment $\iota : \text{Port} \to \text{Opt}$. We write $C(e) = (V, E, \ell, \Sigma, O, t, r, \text{Val})$, omitting ι when clear, where $\text{Val} : V \rightharpoonup \mathcal{V}$ records the value carried by nodes whose signals are determined without reference to variable placeholders (e.g., by ι or by closed values). Placeholders for variables may carry no value, so Val is partial in general. Here $\mathcal{V}$ includes the runtime values above, in particular the semantic optical values $\mathcal{V}_{\text{Opt}}$. All unions below assume node sets are disjoint. Thus for open terms Val may be undefined at some outputs, while for closed well-typed terms Val is defined on all outputs O.

Graph-Compilation with Value-Assignments.

We extend each compilation judgment to produce not just $(V, E, \ell, \Sigma, O, t, r)$ but also a value-assignment $\text{Val} : V \rightharpoonup \mathcal{V}$, relative to the fixed environment ι. Thus each rule has the form

$$\frac{\text{(premises)}}{C(e) = (\,V,\ E,\ \ell,\ \Sigma,\ O,\ t,\ r,\ \text{Val}\,)}$$

Before we define the graph compilation procedure, we introduce some notation that will be convenient. Define $\text{flat}_\tau(v)$ as the left-to-right list of leaf components of a well-typed value $v : \tau$. Formally:

Definition D.2 (Flattening). For each type τ, define $|\tau| \in \mathbb{N}$ and flat_τ by recursion on τ:

$$|b| = 1, \quad \text{flat}_b(v) = (v) \qquad |\tau_1 * \tau_2| = |\tau_1| + |\tau_2|,$$
$$\text{flat}_{\tau_1 * \tau_2}(v_1, v_2) = \text{flat}_{\tau_1}(v_1) \cdot \text{flat}_{\tau_2}(v_2),$$

where b ranges over base types and $\cdot$ denotes concatenation.

A routine induction on the compilation rules establishes the *arity invariant*: if $\Delta; \Pi \vdash e : \tau$ then $C(e)$ has exactly $|\tau|$ ordered outputs, i.e. $|O| = |\tau|$. In particular, for $e_0 : \tau_1 * \tau_2$ the graph $C(e_0)$ has $|O_0| = |\tau_1| + |\tau_2|$ outputs; under the ordering t_0, its first $|\tau_1|$ outputs form the block for the first component and the remaining $|\tau_2|$ form the block for the second. As we will see, C-Unpack wires the placeholders of y_1 onto the first block and those of y_2 onto the second, through the combined index ρ_{y_1, y_2} of the substitution definition, splicing the single graph $C(e_0)$ exactly once. Figures 22 and 23 present the rules for compilation. Each rule produces a graph $(V, E, \ell, \Sigma, O, t, r, \text{Val})$.

The compiler turns each expression into a directed graph that explicitly represents how optical signals flow and where observable outputs appear. It does this compositionally: values and inputs become graphs with a single output node; variables become placeholder output nodes (with no assigned signal value); and compound terms (pairs, phases, splits, unitaries, let, unpack, output) are compiled by wiring together the subgraphs, preserving a left-to-right output ordering. As we build the graph, the Val function serves as the graph semantics. We will show that values computed by a compiled graph are the same as the values computed by the program's small-step semantics.

The set O and ordering t specify the observable outputs of the term; Val assigns values only to nodes whose signals are determined without variable placeholders.

Below, we walk through the rules in Figure 22.

- **C-Pure.** For a base value $v \in \{\mathbb{R}, \text{Unit}, \text{Port}, \text{Unitary}\}$, the graph is a single output node o with $\ell(o) = \bot$, $\Sigma(o) = \bot$, $O = \{o\}$, $t(o) = 1$, and $\text{Val}(o) = v$.
- **C-OptVal.** For a semantic optical value $a \in \mathcal{V}_{\text{Opt}}$, the graph is a single output node o with $\ell(o) = \bot$, $\Sigma(o) = \bot$, $O = \{o\}$, $t(o) = 1$, and $\text{Val}(o) = a$.
- **C-Input.** The input $\texttt{input}(p)$ compiles to one output node o labeled $\texttt{port}(p)$ with $\text{Val}(o) = \iota(p)$.
- **C-Var.** A variable $x : \tau$ compiles to $|\tau|$ placeholder outputs $w_1, \ldots, w_k$ with $\Sigma(w_i) = x$ and Val undefined on these nodes; the leaf indices are recorded by $r(w_i) = i$ and $t(w_i) = i$.
- **C-Pair.** For (e_1, e_2), the graph is the disjoint union of the two subgraphs; outputs are ordered by concatenating the orderings of e_1 and e_2, and Val is the union of the two value maps.
- **C-Phase.** For $\texttt{phase}(\theta, e)$ with a single output o, insert a phase node and a fresh output v, connect $o \to\text{phase}\to v$, and set $\text{Val}(v) = \text{phase}(\theta, \text{Val}(o))$.
- **C-Split.** For $\texttt{split}(r_0, e)$ with a single output o, insert a $\texttt{splitter}(r_0)$ node and two ordered outputs v_1, v_2; set $\text{Val}(v_1) = \text{amp}(\text{Val}(o), r_0)$ and $\text{Val}(v_2) = \text{amp}(\text{Val}(o), 1 - r_0)$.
- **C-Unitary.** For $\texttt{unitary}(U, e)$ with ordered outputs o_1, o_2, insert a unitary node and two ordered outputs w_1, w_2; set $\text{Val}(w_1), \text{Val}(w_2)$ to the components of $U \cdot [\text{Val}(o_1)\ \text{Val}(o_2)]^\top$.

In every rule above, Val is the graph's *semantics*: each node's value is determined by its predecessors through the operation it denotes. We make this notion precise once, because substitution changes edges and must then recompute exactly the values downstream of the changed boundary.

Definition D.3 (Value closure). Fix the input environment ι. For a graph structure (V, E, ℓ, r) and a partial *seed* valuation $B : V \rightharpoonup \mathcal{V}$,

$$\frac{v : b \quad b \in \{\mathbb{R}, \mathsf{Unit}, \mathsf{Port}, \mathsf{Unitary}\} \qquad w \text{ fresh}}{C(v) = (\{w\},\ \emptyset,\ [w \mapsto \bot],\ [w \mapsto \bot],\ \{w\},\ [w \mapsto 1],\ \emptyset,\ [w \mapsto v])}\ \text{C-Pure}$$

$$\frac{a \in \mathcal{V}_{\mathsf{Opt}} \qquad w \text{ fresh}}{C(a) = (\{w\},\ \emptyset,\ [w \mapsto \bot],\ [w \mapsto \bot],\ \{w\},\ [w \mapsto 1],\ \emptyset,\ [w \mapsto a])}\ \text{C-OptVal}$$

$$C(\mathtt{input}(p)) = (\{v\},\ \emptyset,\ [v \mapsto \mathtt{port}(p)],\ [v \mapsto \bot],\ \{v\},\ [v \mapsto 1],\ \emptyset,\ [v \mapsto \iota(p)])\ \text{C-Input}$$

$$\frac{x : \tau \qquad w_1, \ldots, w_k \text{ fresh} \qquad k = |\tau|}{C(x) = (\{w_1, \ldots, w_k\},\ \emptyset,\ [w_i \mapsto \bot]_{i=1}^{k},\ [w_i \mapsto x]_{i=1}^{k},\ \{w_1, \ldots, w_k\},\ [w_i \mapsto i]_{i=1}^{k},\ [w_i \mapsto i]_{i=1}^{k},\ \emptyset)}\ \text{C-Var}$$

$$\frac{C(e_1) = (V_1, E_1, \ell_1, \Sigma_1, O_1, t_1, r_1, \mathsf{Val}_1) \qquad C(e_2) = (V_2, E_2, \ell_2, \Sigma_2, O_2, t_2, r_2, \mathsf{Val}_2)}{C((e_1, e_2)) = \Big(V_1 \cup V_2,\ E_1 \cup E_2,\ \ell_1 \cup \ell_2,\ \Sigma_1 \cup \Sigma_2,\ O_1 \cup O_2,\ [o \mapsto t_1(o)]_{o \in O_1} \cup [o \mapsto |O_1| + t_2(o)]_{o \in O_2},\ r_1 \cup r_2,\ \mathsf{Val}_1 \cup \mathsf{Val}_2\Big)}\ \text{C-Pair}$$

$$\frac{C(e) = (V, E, \ell, \Sigma, O, t, r, \mathsf{Val}) \qquad |O| = 1,\ O = \{o\} \qquad w, v \text{ fresh}}{C(\mathsf{phase}(\theta, e)) = \Big(V \cup \{w, v\},\ E \cup \{(o, w), (w, v)\},\ \ell \cup [w \mapsto \mathsf{phase}(\theta)],\ v \mapsto \bot],\ \Sigma \cup [w \mapsto \bot, v \mapsto \bot],\ \{v\},\ [v \mapsto 1],\ r,\ \mathsf{Val} \cup [\,w \mapsto \mathsf{phase}(\theta, \mathsf{Val}(o)),\ v \mapsto \mathsf{phase}(\theta, \mathsf{Val}(o))\,]\Big)}\ \text{C-Phase}$$

$$\frac{C(e) = (V, E, \ell, \Sigma, O, t, r, \mathsf{Val}) \qquad |O| = 1,\ O = \{o\} \qquad w, v_1, v_2 \text{ fresh}}{\begin{array}{l} C(\mathsf{split}(r_0, e)) = \Big(V \cup \{w, v_1, v_2\},\ E \cup \{(o, w), (w, v_1), (w, v_2)\},\ \ell \cup [w \mapsto \mathtt{splitter}(r_0),\ v_1 \mapsto \bot,\ v_2 \mapsto \bot], \\ \Sigma \cup [w \mapsto \bot, v_1 \mapsto \bot, v_2 \mapsto \bot],\ \{v_1, v_2\},\ [v_1 \mapsto 1,\ v_2 \mapsto 2],\ r \cup [v_1 \mapsto 1,\ v_2 \mapsto 2], \qquad \mathsf{Val} \cup [\,v_1 \mapsto \mathsf{amp}(\mathsf{Val}(o),\ r_0),\ v_2 \mapsto \mathsf{amp}(\mathsf{Val}(o),\ 1 - r_0)\,]\Big) \end{array}}\ \text{C-Split}$$

$$\frac{C(e) = (V, E, \ell, \Sigma, O, t, r, \mathsf{Val}) \qquad w, w_1, w_2 \text{ fresh} \qquad |O| = 2,\ O = \{o_1, o_2\},\ t(o_1) = 1,\ t(o_2) = 2}{\begin{array}{l} C(\mathsf{unitary}(U, e)) = \Big(V \cup \{w, w_1, w_2\},\ E \cup \{(o_1, w), (o_2, w), (w, w_1), (w, w_2)\},\ \ell \cup [w \mapsto U,\ w_1 \mapsto \bot,\ w_2 \mapsto \bot], \\ \Sigma \cup [w \mapsto \bot,\ w_1 \mapsto \bot,\ w_2 \mapsto \bot],\ \{w_1, w_2\},\ [w_1 \mapsto 1,\ w_2 \mapsto 2],\ r \cup [w_1 \mapsto 1,\ w_2 \mapsto 2], \qquad \mathsf{Val} \cup [\,w_1 \mapsto (U \cdot [\mathsf{Val}(o_1) \quad \mathsf{Val}(o_2)])[0],\ w_2 \mapsto (U \cdot [\mathsf{Val}(o_1) \quad \mathsf{Val}(o_2)])[1]\,]\Big) \end{array}}\ \text{C-Unitary}$$

$$\frac{C(e_1) = (V_1, E_1, \ell_1, \Sigma_1, O_1, t_1, r_1, \mathsf{Val}_1) \qquad C(e_2) = (V_2, E_2, \ell_2, \Sigma_2, O_2, t_2, r_2, \mathsf{Val}_2) \qquad |O_1| = 1,\ O_1 = \{o\} \qquad v_O \text{ fresh}}{C(\mathsf{output}(p) \leftarrow e_1; e_2) = \Big(V_1 \cup V_2 \cup \{v_O\},\ E_1 \cup E_2 \cup \{(o, v_O)\},\ \ell_1 \cup \ell_2 \cup [v_O \mapsto \mathtt{output}(p)],\ \Sigma_1 \cup \Sigma_2 \cup [v_O \mapsto \bot],\ O_2, \qquad t_2,\ r_1 \cup r_2,\ \mathsf{Val}_1 \cup \mathsf{Val}_2 \cup [v_O \mapsto \mathsf{Val}_1(o)]\Big)}\ \text{C-Out}$$

Figure 22: Compilation Rules for Primitives without variable binding

$$\frac{C(e_1) = G_1 \quad C(e_2) = G_2}{C(\mathtt{let}\ x = e_1\ \mathtt{in}\ e_2) = G_2[x \mapsto G_1]}\ \text{C-Let} \qquad \frac{C(e_0) = G_0, \quad C(e_2) = G_2}{C(\mathtt{unpack}\ e_0\ \mathtt{as}\ (y_1, y_2)\ \mathtt{in}\ e_2) \ = \ G_2[(y_1, y_2) \mapsto G_0]}\ \text{C-Unpack}$$

Figure 23: Compilation Rules for Primitives with variable binding

write

$$\mathsf{close}^{\iota}_{E,\ell,r}(B)$$

for the least partial valuation $\mathsf{Val} : V \rightharpoonup \mathcal{V}$ satisfying $B \subseteq \mathsf{Val}$ and the following value equations whenever their right-hand sides are defined:

$$\ell(v) = \mathtt{port}(p) \Longrightarrow \mathsf{Val}(v) = \iota(p),$$

$$(u, w), (w, z) \in E,\ \ell(w) = \mathsf{phase}(\theta) \Longrightarrow \mathsf{Val}(w) = \mathsf{Val}(z) = \mathsf{phase}(\theta, \mathsf{Val}(u)),$$

$$(u, w), (w, z_i) \in E,\ \ell(w) = \mathtt{splitter}(r_0),\ r(z_i) = i \Longrightarrow \begin{cases} \mathsf{Val}(z_1) = \mathsf{amp}(\mathsf{Val}(u), r_0), \\ \mathsf{Val}(z_2) = \mathsf{amp}(\mathsf{Val}(u), 1 - r_0), \end{cases}$$

$$\ell(w) = U,\ u_i \text{ is input } i \text{ of } w,\ (w, z_i) \in E,\ r(z_i) = i \Longrightarrow (\mathsf{Val}(z_1), \mathsf{Val}(z_2)) = U \cdot [\mathsf{Val}(u_1)\ \mathsf{Val}(u_2)]^{\top},$$

$$(u, w) \in E,\ \ell(w) = \mathtt{output}(p) \Longrightarrow \mathsf{Val}(w) = \mathsf{Val}(u).$$

The ordered inputs of a unitary node are the two inputs fixed when the compilation rule creates the node; substitution below preserves those input positions when it redirects an edge. For finite acyclic compiled graphs this least valuation is unique and is computed by one topological forward pass. Nodes that transitively depend on a seedless placeholder remain undefined. When the graph structure is clear, we write simply $\mathsf{close}(B)$. Each compilation rule of Figures 22–23 sets $\mathsf{Val} = \mathsf{close}(S)$, where S records the stored constants of pure and optical-value nodes; $\mathtt{port}(p)$ nodes are supplied by the first equation.

Before explaining the compilation rules for terms with variable binding, we define the graph operation used by both C-Let and C-Unpack. It is useful to separate the primitive operation, called a *splice*, from the two syntactic substitution forms that instantiate it.

Definition D.4 (Graph splice). Let

$$G_1 = (V_1, E_1, \ell_1, \Sigma_1, O_1, t_1, r_1, \mathsf{Val}_1),$$
$$G_2 = (V_2, E_2, \ell_2, \Sigma_2, O_2, t_2, r_2, \mathsf{Val}_2),$$

with $V_1 \cap V_2 = \emptyset$. Let $P \subseteq V_2$ be a set of placeholder nodes of G_2, so every $v \in P$ satisfies $\ell_2(v) = \bot$ and $\Sigma_2(v) \neq \bot$. A map $\rho : P \to \{1, \ldots, |O_1|\}$ is an *admissible boundary map* when it is a bijection and each $v \in P$ is a source with either no outgoing edge ($v \in O_2$) or exactly one outgoing edge ($v \notin O_2$). For such ρ, let

$$o_\rho(v) \quad \text{be the unique } o \in O_1 \text{ such that } t_1(o) = \rho(v).$$

Let

$$R_P = \{w \in V_2 \setminus P \mid \exists v \in P.\ \text{there is a nonempty directed path in } E_2 \text{ from } v \text{ to } w\}.$$

The splice $G_2[P, \rho \mapsto G_1]$ is the graph $(V', E', \ell', \Sigma', O', t', r', \mathsf{Val}')$ defined by:

$$
\begin{aligned}
P^O &= P \cap O_2,\\
V' &= (V_2 \setminus P) \ \cup \ V_1,\\
E' &= \{(u,v) \in E_2 \mid u \notin P,\ v \notin P\} \ \cup \ E_1\\
&\quad \cup \ \{(o_\rho(v), v') \mid v \in P,\ (v,v') \in E_2\},\\
\ell'(w) &= \begin{cases} \ell_1(w), & w \in V_1,\\ \ell_2(w), & w \in V_2 \setminus P, \end{cases} \qquad \Sigma'(w) = \begin{cases} \Sigma_1(w), & w \in V_1,\\ \Sigma_2(w), & w \in V_2 \setminus P, \end{cases}\\
O' &= (O_2 \setminus P) \ \cup \ \{\, o_\rho(v) \mid v \in P^O \,\},\\
t'(o) &= \begin{cases} t_2(o), & o \in O_2 \setminus P,\\ t_2(v), & o = o_\rho(v) \text{ for some } v \in P^O, \end{cases}\\
r'(v) &= \begin{cases} r_1(v), & v \in V_1,\\ r_2(v), & v \in V_2 \setminus P, \end{cases}\\
B' &= \mathsf{Val}_1 \cup (\mathsf{Val}_2 \restriction_{V_2 \setminus (P \cup R_P)}),\\
\mathsf{Val}' &= \mathsf{close}^{\iota}_{E',\ell',r'}(B').
\end{aligned}
$$

Thus every placeholder $v \in P$ is removed, and its outgoing edge, if any, is redirected from v to the matching output $o_\rho(v)$ of G_1. If the placeholder was itself an expression output of G_2, that output position is transferred to $o_\rho(v)$. The inserted graph G_1 appears once as a whole: the node set V_1 is included exactly once, so internal nodes and $\texttt{output}(p)$ nodes of G_1 are preserved rather than copied per placeholder. The valuation is then recomputed by closing the values of G_1 and the values of body nodes not reachable from the deleted placeholders; nodes in R_P are intentionally not used as seeds because their inputs may have changed.

Definition D.5 (Graph substitution instances). The compilation rules use only the following two instances of Definition D.4.

Single variable. For a variable x, let

$$P_x = \{v \in V_2 \mid \Sigma_2(v) = x\}.$$

When $r_2|_{P_x} : P_x \to \{1, \ldots, |O_1|\}$ is an admissible boundary map, define

$$G_2[x \mapsto G_1] \ := \ G_2[P_x, r_2|_{P_x} \mapsto G_1].$$

Pair unpacking. For variables $y_1 : \tau_1$ and $y_2 : \tau_2$, let

$$P_{y_i} = \{v \in V_2 \mid \Sigma_2(v) = y_i\}$$

and define

$$\rho_{y_1,y_2}(v) = \begin{cases} r_2(v), & v \in P_{y_1},\\ |\tau_1| + r_2(v), & v \in P_{y_2}. \end{cases}$$

When $\rho_{y_1,y_2} : P_{y_1} \cup P_{y_2} \to \{1, \ldots, |O_1|\}$ is an admissible boundary map, define

$$G_2[(y_1, y_2) \mapsto G_1] \ := G_2[P_{y_1} \cup P_{y_2}, \rho_{y_1,y_2} \mapsto G_1].$$

For well-typed C-Let and C-Unpack uses, admissibility follows from Lemma C.1, C-Var, and the arity invariant: $|P_x| = |\tau_x| = |O_1|$ for single-variable substitution, and $|P_{y_1}| + |P_{y_2}| = |\tau_1| + |\tau_2| = |O_1|$ for pair unpacking.

A splice is oblivious to the internal structure of the inserted graph except for two observable interfaces: its ordered expression outputs, which feed the body, and its internal $\texttt{output}(p)$ nodes, which are retained as side effects. The next two lemmas state the structural and semantic facts used later.

Lemma D.6 (Splice preserves well-formedness). *Let G_1 and G_2 be well-formed graphs with disjoint node sets and with no concrete port label appearing in both graphs. If ρ is an admissible boundary map for $P \subseteq V_2$, then $G_2[P, \rho \mapsto G_1]$ is well-formed.*

Proof Sketch. Labels and variable assignments are inherited from G_1 and $G_2 \setminus P$, so their domains and the implication $\Sigma(v) \neq \bot \implies \ell(v) = \bot$ are immediate. For degrees, deleting P removes only source placeholders. A body successor v' of a deleted placeholder loses the edge (v, v') and gains exactly the replacement edge $(o_\rho(v), v')$, so its in-degree is unchanged. Nodes of G_1 keep their internal edges. Each output $o \in O_1$ is $o_\rho(v)$ for a unique $v \in P$. If $v \in O_2$, then v had no outgoing edge and o becomes an output of the splice with out-degree 0. If $v \notin O_2$, then v had one outgoing edge and o gains exactly one outgoing replacement edge and is not in O'. Thus every degree condition is preserved. The output set replaces each $v \in P \cap O_2$ by the unique $o_\rho(v)$ and leaves all other body outputs unchanged; assigning $t'(o_\rho(v)) = t_2(v)$ therefore preserves bijectivity of the output ordering. The map r' is the disjoint union of the two existing r-maps. Finally, port uniqueness follows from port uniqueness in each graph and the assumed disjointness of their concrete port labels. □

Lemma D.7 (Splice invariance). *Fix G_2, P, and an admissible boundary map ρ. For every node $w \in V_2 \setminus P$, the value assigned to w in $G_2[P, \rho \mapsto G_1]$, when defined, is a fixed function of G_2, ρ, and the tuple* $\mathsf{out}(G_1)$. *Consequently, if* $\mathsf{out}(G_1) = \mathsf{out}(G_1')$, *then*

$$\mathsf{out}\big(G_2[P, \rho \mapsto G_1]\big) = \mathsf{out}\big(G_2[P, \rho \mapsto G_1']\big),$$

and the two results assign the same value to every $\texttt{output}(p)$ *node inherited from G_2.*

Proof Sketch. All edges from G_1 into the body are replacement edges $(o_\rho(v), v')$ for former placeholder edges (v, v'). The value delivered across such an edge is the $\rho(v)$-th component of $\mathsf{out}(G_1)$. No other internal node of G_1 has an edge into $V_2 \setminus P$. The splice valuation on body nodes is obtained by close from two kinds of seeds: the unchanged body seeds outside $P \cup R_P$, which are determined by G_2, and the boundary values just listed, which are determined by $\mathsf{out}(G_1)$ and ρ. Acyclicity then makes the resulting value of each body node the unique result of propagating the fixed value equations in topological order. Therefore replacing G_1 by any G_1' with the same ordered output tuple leaves all body-node values unchanged. Expression outputs inherited from G_2 are body nodes, and the predecessors of $\texttt{output}(p)$ nodes inherited from G_2 are body nodes, so the two stated consequences follow. □

Next, we show that well-typed terms are well-formed graphs. We define well-formed graphs as follows:

Definition D.8 (Well Formed Graphs). A compiled graph

$$G = (V, E, \ell, \Sigma, O, t, r, \mathsf{Val})$$

is *well-formed* if all of the following hold:

1. Label-domain: $\ell : V \to \{\texttt{port}(p), \texttt{output}(p), \texttt{phase}(\theta),$ $, \texttt{splitter}(r), U \in \mathsf{Unitary}, \bot\}$.

2. Variable-assignment: $\Sigma : V \to \mathsf{Vars} \cup \{\bot\}$, and whenever $\Sigma(v) \neq \bot$, then $\ell(v) = \bot$.

3. Degree constraints:

If $\ell(v) = \texttt{port}(p)$ or $\Sigma(v) \neq \bot$,

$$indeg(v) = 0, \quad outdeg(v) = \begin{cases} 0, & v \in O, \\ 1, & v \notin O. \end{cases}$$

If $\ell(v) = \texttt{phase}(\theta)$, $indeg(v) = 1$, $outdeg(v) = 1$.

If $\ell(v) = \texttt{splitter}(r)$, $indeg(v) = 1$, $outdeg(v) = 2$.

If $\ell(v) = U \in \text{Unitary}$, $indeg(v) = 2$, $outdeg(v) = 2$.

If $\ell(v) = \texttt{output}(p)$, $indeg(v) = 1$, $outdeg(v) = 0$.

If $v \in O$, $outdeg(v) = 0$.

4. Output-ordering: $t : O \to \{1, \ldots, |O|\}$ is a bijection, and r is defined so that every output of a splitter or unitary is matched to the correct index in $\{1, 2\}$, and every placeholder node carries its leaf index.

5. Port uniqueness: for every concrete port p, there is at most one node v whose label is either $\texttt{port}(p)$ or $\texttt{output}(p)$.

The out-degree 0 case for a $\texttt{port}(p)$ node is an intermediate compilation state: the node is an expression output whose signal has not yet been consumed by an enclosing component. Once that signal is consumed, the node leaves O and has out-degree 1. Thus fully routed circuits satisfy the usual physical-input-port out-degree condition. For fully routed circuits, the present definition (Definition D.8) entails Definition 3.1: it includes the degree constraints, with the intermediate $v \in O$ case discharged by routing, and it adds the port-uniqueness condition. For physical realizability these degree conditions are sufficient. However, here we will work with this definition that captures complete well-formedness for the compiler. We now prove our main safety theorem. Graphs of well-typed terms are well-formed.

Theorem D.9 (Graphs of Well-Typed Terms Are Well-Formed). *If $\Delta; \Pi \vdash e : \tau$ is derivable, then $C(e)$ is a well-formed graph.*

Proof. By induction on the typing derivation $\Delta; \Pi \vdash e : \tau$.

Case (T-Input). $e = \texttt{input}(p)$, with exact context $\emptyset; p : \text{Port}$. C-Input produces a single node v with $\ell(v) = \texttt{port}(p)$, $\Sigma(v) = \bot$, $O = \{v\}$, $\text{Val}(v) = \iota(p)$, and no edges. Hence $indeg(v) = 0$ and, since $v \in O$, $outdeg(v) = 0$. All other constraints hold vacuously.

Case (T-Unit, T-Const, T-U-Const, T-Port, T-OptVal). e is a value $v \in \{r, (), p, U\}$ or a semantic optical value $a \in \mathcal{V}_{\text{Opt}}$; in the T-Port case the exact context is $\emptyset; p : \text{Port}$. C-Pure or C-OptVal yields a single output node v_o with $\ell(v_o) = \bot$, $\Sigma(v_o) = \bot$, and no edges. Thus $indeg(v_o) = 0$ and $outdeg(v_o) = 0$; all other conditions are immediate.

Case (T-Var). $e = x$, with exact context $x : \tau; \emptyset$. C-Var produces $k = |\tau|$ placeholder nodes $w_1, \ldots, w_k$ with $\Sigma(w_i) = x$, $\ell(w_i) = \bot$, no edges, and $O = \{w_1, \ldots, w_k\}$ ordered by $t(w_i) = i$. Thus each $indeg(w_i) = 0$ and $outdeg(w_i) = 0$, and the ordering conditions hold by construction.

Case (T-Phase). $e = \texttt{phase}(\theta, e_0)$. By IH $C(e_0)$ is well-formed, with exactly one output $o \in O_0$. C-Phase adds fresh nodes w, v with edges (o, w) and (w, v), $\ell(w) = \texttt{phase}(\theta)$, $\ell(v) = \bot$, $indeg(w) = 1$, $outdeg(w) = 1$, and $v \in O$ with $outdeg(v) = 0$. All other nodes keep their degrees. Output-ordering is trivial since $|O| = 1$. Thus well-formed.

Case (T-Split). $e = \texttt{split}(r_0, e_0)$. By IH $O_0 = \{o\}$. C-Split adds fresh nodes w, v_1, v_2 with edges (o, w), (w, v_1), (w, v_2), $\ell(w) = \texttt{splitter}(r_0)$, $indeg(w) = 1$, $outdeg(w) = 2$, and $O = \{v_1, v_2\}$, $t(v_1) = 1, t(v_2) = 2$. All constraints satisfied.

Case (T-Unitary).

$$\frac{\emptyset; \emptyset \vdash U : Unitary \qquad \Delta_1; \Pi_1 \vdash e_1 : \text{Opt} \qquad \Delta_2; \Pi_2 \vdash e_2 : \text{Opt}}{\Delta_1, \Delta_2; \Pi_1, \Pi_2 \vdash \texttt{unitary}(U, (e_1, e_2)) : (\text{Opt} * \text{Opt})}\ \text{T-Unitary}$$

By IH, $C(e_1)$ and $C(e_2)$ are well-formed. Let $e_0 = (e_1, e_2)$; then C-Pair yields a well-formed graph $C(e_0)$ with outputs $O_0 = \{o_1, o_2\}$ ordered by $t(o_1) = 1, t(o_2) = 2$. C-Unitary adds fresh w, w_1, w_2 and edges (o_1, w), (o_2, w), (w, w_1), (w, w_2), with $\ell(w) = U$, $indeg(w) = 2$, $outdeg(w) = 2$. The new $O = \{w_1, w_2\}$ satisfies $outdeg(w_i) = 0$, $t(w_1) = 1, t(w_2) = 2$, and $r(w_1) = 1, r(w_2) = 2$. All other nodes preserve their degrees. Thus well-formed.

Case (T-Pair). (e_1, e_2). By IH each $C(e_i)$ is well-formed. C-Pair unions graphs, so degrees are unchanged, and $O = O_1 \cup O_2$. The new ordering t agrees with t_1 on O_1 and offsets t_2 by $|O_1|$ on O_2, hence is a bijection on O. Therefore the resulting graph is well-formed.

Case (T-LetLin).

$$\frac{\Delta_1; \Pi_1 \vdash e_1 : \tau_1 \qquad \Delta_2,\, x : \tau_1; \Pi_2 \vdash e_2 : \tau}{\Delta_1 \cup \Delta_2; \Pi_1 \cup \Pi_2 \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau}\ \text{T-LetLin}$$

By IH let $C(e_1) = G_1 = (V_1, E_1, \ell_1, \Sigma_1, O_1, t_1, r_1, \text{Val}_1)$ and $C(e_2) = G_2 = (V_2, E_2, \ell_2, \Sigma_2, O_2, t_2, r_2, \text{Val}_2)$ be well-formed. We must show

$$C\big(\texttt{let}\ x = e_1\ \texttt{in}\ e_2\big) = G_2[x \mapsto G_1]$$

is well-formed. Since $\Delta_2, x : \tau_1; \Pi_2 \vdash e_2 : \tau$, Lemma C.1 gives $occ(x, e_2) = 1$. Therefore

$$P_x = \{v \in V_2 \mid \Sigma_2(v) = x\}$$

is exactly the $|\tau_1|$-node placeholder block created for this one occurrence of x, and $r_2|_{P_x}$ enumerates it by $\{1, \ldots, |\tau_1|\}$. The arity invariant gives $|O_1| = |\tau_1|$, so $r_2|_{P_x} : P_x \to \{1, \ldots, |O_1|\}$ is a bijection. Since G_2 is well-formed, every node in P_x is a source placeholder with out-degree 0 if it is an expression output and out-degree 1 otherwise. Thus $r_2|_{P_x}$ is an admissible boundary map. The concrete port labels in the two premise graphs are disjoint because Π_1 and Π_2 are combined by disjoint union and concrete port labels are introduced only by C-Input and C-Out. Hence Lemma D.6 applies, and $G_2[x \mapsto G_1]$ is well-formed.

Case (T-UnpackLin):

$$\frac{\Delta_0; \Pi_0 \vdash e_0 : (\tau_1, \tau_2) \qquad \Delta_2, y_1 : \tau_1, y_2 : \tau_2; \Pi_2 \vdash e_2 : \tau}{\Delta_0 \cup \Delta_2; \Pi_0 \cup \Pi_2 \vdash \texttt{unpack}\ e_0\ \texttt{as}\ (y_1, y_2)\ \texttt{in}\ e_2 : \tau}\ \text{T-UnpackLin}$$

By IH $C(e_0) = G_0$ and $C(e_2) = G_2$ are well-formed, and by C-Unpack $C(e) = G_2[(y_1, y_2) \mapsto G_0] = G_2[P_{y_1} \cup P_{y_2},\ \rho_{y_1,y_2} \mapsto G_0]$. Since $\Delta_2, y_1 : \tau_1, y_2 : \tau_2; \Pi_2 \vdash e_2 : \tau$, Lemma C.1 gives $occ(y_i, e_2) = 1$; hence by C-Var each placeholder set P_{y_i} has exactly $|\tau_i|$ nodes, with r_2 enumerating $\{1, \ldots, |\tau_i|\}$. Together with the arity invariant $|O_0| = |\tau_1| + |\tau_2|$, the combined index ρ_{y_1,y_2} is a bijection $P_{y_1} \cup$

$P_{y_2} \to \{1, \ldots, |O_0|\}$. Well-formedness of G_2 gives the source/out-degree requirements for all placeholders in $P_{y_1} \cup P_{y_2}$, so this is an admissible boundary map. The concrete port labels of G_0 and G_2 are disjoint because the premise port contexts Π_0 and Π_2 are disjoint. Lemma D.6 then shows that the single splice $G_2[P_{y_1} \cup P_{y_2}, \rho_{y_1,y_2} \mapsto G_0]$ is well-formed. Hence $C(e)$ is well-formed.

Case (T-Out). Suppose $\Delta_1; \Pi_1 \vdash e_1 : \text{Opt}$ and $\Delta_2; \Pi_2 \vdash e_2 : \tau$, so $\texttt{output}(p) \leftarrow e_1; e_2$ is well-typed. By IH, $C(e_1)$ and $C(e_2)$ are well-formed, and O_1 is a singleton. Rule C-Out adds a fresh node v_o with label $\texttt{output}(p)$ and a single incoming edge from the unique output of $C(e_1)$; thus $\mathit{indeg}(v_o) = 1$, $\mathit{outdeg}(v_o) = 0$, and $v_o \notin O$. All other nodes and the output ordering t of $C(e_2)$ are unchanged. Hence the resulting graph is well-formed.

Port uniqueness. The port-uniqueness condition is preserved in all cases by the linear port context. The only compilation rules that introduce concrete port labels are C-Input, which introduces one $\texttt{port}(p)$, and C-Out, which introduces one $\texttt{output}(p)$. In the typing rules, each such p : Port is a singleton resource, and compound rules combine port contexts only by disjoint union. Therefore two subgraphs combined by C-Pair, C-Let, C-Unpack, C-Phase, C-Split, C-Unitary, or C-Out cannot contain the same concrete port label. Graph substitution replaces variable placeholders only; it does not duplicate or identify concrete $\texttt{port}(p)$ or $\texttt{output}(p)$ nodes. Hence no compiled graph contains two nodes labelled with the same concrete input/output port p.

This completes all cases. Therefore, every $C(e)$ is well-formed. □

D.1 Correctness of the Compiler

Next, we show that optical values routed to ports by a λ_λ program are the same as those routed by its compiled photonic circuit graph. To show this we establish a series of lemmas. We first pin down which nodes of a compiled graph are placeholders, and use this to show that closed programs—and, more generally, fully substituted graphs—assign a value to every output.

Lemma D.10 (Placeholders are the free-variable leaves). *If $\Delta; \Pi \vdash e : \tau$ and $C(e) = (V, E, \ell, \Sigma, O, t, r, \text{Val})$, then the variable-carrying nodes of $C(e)$ are exactly the placeholders of its free variables,*

$$\{v \in V \mid \Sigma(v) \neq \bot\} = \bigcup_{x:\tau_x \in \Delta} P_x, \qquad P_x = \{v \in V \mid \Sigma(v) = x\},$$

and for each $x : \tau_x \in \Delta$ we have $|P_x| = |\tau_x|$, with r restricting to a bijection $P_x \to \{1, \ldots, |\tau_x|\}$. In particular, if e is closed then $C(e)$ has no placeholder node.

Proof. Induction on the typing derivation. C-Var creates exactly the $|\tau|$ placeholders of its sole free variable, indexed by r. C-Pure, C-OptVal, and C-Input create no variable-carrying node and have empty variable context. The compositional rules C-Pair, C-Phase, C-Split, C-Unitary, and C-Out add only $\bot$-labeled nodes with $\Sigma = \bot$, and their variable contexts are the disjoint unions of the premises', so the statement is preserved. For C-Let the bound variable x leaves the context, and correspondingly graph substitution deletes exactly the placeholders P_x (Lemma C.1 gives $\mathit{occ}(x, e_2) = 1$, so $|P_x| = |\tau_1|$), leaving the placeholders of $\Delta_1 \cup \Delta_2$; C-Unpack is identical for y_1, y_2. No rule creates a placeholder for a variable outside its context. □

Lemma D.11 (Placeholder-free graphs have valued observables). *Call a graph* placeholder-free *if $\Sigma(v) = \bot$ for every node v. If a graph $G = (V, E, \ell, \Sigma, O, t, r, \text{Val})$ built by the compilation rules (Figures 22–23) is placeholder-free, then* Val *is defined on every expression output $o \in O$. Moreover, if $\ell(u) = \texttt{output}(p)$ and $(v, u) \in E$, then $\text{Val}(v)$ and $\text{Val}(u)$ are defined.*

Proof Sketch. Acyclicity holds by construction: each compilation rule directs its new edges from already-present outputs into fresh nodes, and graph substitution only redirects a placeholder's out-edge (v, v') to (o, v') for an output o of the inserted graph—which sends edges out of its outputs but receives none, so no cycle is formed. In a placeholder-free graph every source (in-degree-0 node) is a $\texttt{port}(p)$, pure, or optical-value node, each of which close seeds with a value ($\iota(p)$, or the stored constant). Taking the nodes in topological order, every signal output of a phase, splitter, or unitary whose predecessor signals are already valued is assigned the corresponding value by close. Similarly, every $\texttt{output}(p)$ node whose predecessor is valued is assigned that predecessor's value.

Thus every signal-carrying node reachable from a source has a value. The nodes in O are exactly expression-output signal nodes, and the predecessor of any $\texttt{output}(p)$ node is also such a signal node. Hence all expression outputs and all port-output predecessors are valued; then the final $\texttt{output}(p)$ nodes are valued as well. Internal splitter and unitary component nodes need not themselves carry scalar optical values, which is why the claim is restricted to observables. □

Corollary D.12 (Closed and fully substituted terms have valued observables). (i) *If $\emptyset; \Pi \vdash e : \tau$, then* Val *is defined on all outputs of $C(e)$, and on the predecessor of every $\texttt{output}(p)$ node in $C(e)$.* (ii) *If $\Delta; \Pi \vdash e : \tau$ and G is obtained from $C(e)$ by splicing, into each free-variable placeholder set P_x, a placeholder-free compiled graph with matching output arity then* Val *is defined on all outputs of G, and on the predecessor of every $\texttt{output}(p)$ node in G.*

Proof. By Lemma D.10, in case (i) a closed term's graph has no placeholder. In case (ii), every placeholder of $C(e)$ belongs to one of the free-variable sets P_x, and the splices remove all of those placeholders. The inserted graphs are placeholder-free, so the resulting graph is placeholder-free. For $G = C(e)[\rho_\gamma]$, each inserted graph is $C(\gamma(x))$; since $\gamma(x)$ is a value, its graph has no placeholder nodes. In both cases Lemma D.11 applies. □

Output tuples. We define a notion of output tuples that helpful in expressing the output values of a graph. For a graph $G = (V, E, \ell, \Sigma, O, t, r, \text{Val})$ such that Val is defined on all outputs O, write $\text{out}(G) = (\text{Val}(o_1), \ldots, \text{Val}(o_k))$ where $t(o_i) = i$. When $k = 1$, we identify $\text{out}(G)$ with the single value. When $k = 0$, $\text{out}(G)$ is the empty tuple.

Observable Port Outputs. For a graph $G = (V, E, \ell, \Sigma, O, t, r, \text{Val})$, define the partial map $\text{portout}(G) : \text{Port} \rightharpoonup \text{Opt}$ as follows. For each port p, if there exists a node $u \in V$ with $\ell(u) = \texttt{output}(p)$, let v be its unique predecessor (well-formedness gives $\mathit{indeg}(u) = 1$); set $\text{portout}(G)(p) = \text{Val}(v)$. If no such node exists, or if $\text{Val}(v)$ is undefined, then $\text{portout}(G)(p)$ is undefined. For well-typed terms, linearity ensures at most one $\texttt{output}(p)$ node for each p, so the map is well-defined.

We can now state compiler correctness. Rather than argue only about closed programs, we prove a single statement about *open* terms under a *closing substitution*. This lets us induct on the typing derivation. Otherwise, we would have had to induct on the number of evaluation steps.

Definition D.13 (Shape of a value). A value w is *τ-shaped* if $\mathsf{flat}_\tau(w)$ is defined, i.e. w matches the product structure of τ: a base type is matched by a value of that base sort (in particular, an Opt-shaped value is any $a \in \mathcal{V}_{\mathsf{Opt}}$), and $\tau_1 * \tau_2$ is matched by a pair (w_1, w_2) with each w_i being τ_i-shaped. If w is τ-shaped then $\mathsf{flat}_\tau(w)$ is a tuple of length $|\tau|$, and (by induction on w, via C-Pure, C-OptVal, C-Pair) $C(w)$ has $|\tau|$ ordered outputs with $\mathsf{out}(C(w)) = \mathsf{flat}_\tau(w)$ and no $\mathtt{output}(p)$ node.

Definition D.14 (Closing substitution and its graph action). Let $\Delta; \Pi \vdash e : \tau$. A *closing substitution* for e is a map γ assigning a τ_x-shaped value $\gamma(x)$ to each $x : \tau_x \in \Delta$. We write $e\gamma$ for the closed term obtained by the capture-avoiding replacement of every free x by $\gamma(x)$; and, for $\Delta' \subseteq \Delta$, $\gamma|_{\Delta'}$ for the restriction of γ to the variables of Δ'. The *graph action* of γ on $C(e)$ is

$$C(e)[\rho_\gamma] \;:=\; C(e)\,[\,x_1 \mapsto C(\gamma(x_1))\,]\cdots [\,x_n \mapsto C(\gamma(x_n))\,], \qquad \{x_1,\dots,x_n\} = \mathrm{dom}(\Delta).$$

the single-variable splice of the value graph $C(\gamma(x_i))$ into the placeholder set P_{x_i} of each free variable. This is well defined: by Lemma D.10 each P_{x_i} has $|\tau_{x_i}|$ nodes, and by Definition D.13 $C(\gamma(x_i))$ has $|\tau_{x_i}|$ outputs; the splices act on pairwise disjoint placeholder sets and insert graphs with no $\mathtt{output}(p)$ node, so the result is independent of their order.

Lemma D.15 (Closing-Substitution Correctness). *Let $\Delta; \Pi \vdash e : \tau$, let γ be a closing substitution for e (Definition D.14), and let ι be an input environment defined on every port occurring in $e\gamma$. If $(e\gamma, \sigma_0) \to_\iota^* (v, \sigma')$ with $\sigma_0 = \emptyset$, then v is τ-shaped and*

$$\textit{(i)}\quad \mathsf{out}\big(C(e)[\rho_\gamma]\big) = \mathsf{flat}_\tau(v), \qquad \textit{(ii)}\quad \mathsf{portout}\big(C(e)[\rho_\gamma]\big) = \sigma'.$$

Proof Sketch. By induction on the typing derivation of $\Delta; \Pi \vdash e : \tau$. We use four standing facts. *(a)* No reduction rule inspects the store σ, so both the value and the set of port-writes produced by an evaluation are independent of the initial store; we may therefore run every sub-evaluation from $\sigma_0 = \emptyset$. *(b)* By port-linearity, subterms drawn from disjoint port contexts write to disjoint ports, so their stores combine by disjoint union. *(c)* For any value w, Definition D.13 gives $\mathsf{out}(C(w)) = \mathsf{flat}(w)$, with $C(w)$ free of $\mathtt{output}(p)$ nodes. *(d)* Every graph to which $\mathsf{out}(\cdot)$ or $\mathsf{portout}(\cdot)$ is applied below is fully substituted, hence placeholder-free, so by Corollary D.12(ii) its outputs are all valued and every equation below is between well-defined tuples. We abbreviate $\gamma_i = \gamma|_{\Delta_i}$ (Definition D.14).

Case (T-Var). $e = x$, $\Delta = \{x : \tau\}$, and $\gamma(x) = v$ is τ-shaped. Then $e\gamma = v$ is already a value, so it reduces in zero steps to itself: v is τ-shaped and $\sigma' = \sigma_0 = \emptyset$. The placeholder graph $C(x)$ is spliced to $C(v)$, so $C(e)[\rho_\gamma] = C(x)[x \mapsto C(v)] = C(v)$. By fact (c), $\mathsf{out}(C(e)[\rho_\gamma]) = \mathsf{flat}_\tau(v)$ and $\mathsf{portout}(C(e)[\rho_\gamma]) = \emptyset = \sigma'$.

Cases (T-Const), (T-Unit), (T-U-Const), (T-OptVal). $\Delta = \emptyset$, γ is empty, and $e = v$ is a base or optical value. Then $e\gamma = v$ reduces in zero steps, $\sigma' = \emptyset$, and $C(e)[\rho_\gamma] = C(v)$; fact (c) gives both equations, and v is τ-shaped.

Case (T-Input). $e = \mathtt{input}(p)$, $\Delta = \emptyset$. Then $e\gamma \to_\iota \iota(p) = v$, an Opt-shaped value, with $\sigma' = \emptyset$. C-Input gives $\mathsf{out}(C(e)) = (\iota(p)) = \mathsf{flat}_{\mathsf{Opt}}(v)$ and no $\mathtt{output}(p)$ node, so $\mathsf{portout}(C(e)) = \emptyset$.

Case (T-Pair). $e = (e_1, e_2)$, $\Delta = \Delta_1, \Delta_2$; split $\gamma = \gamma_1 \uplus \gamma_2$. Evaluation of $e\gamma$ is left-to-right, giving $(e_1\gamma_1, \sigma_0) \to^* (v_1, \sigma_1)$, $(e_2\gamma_2, \sigma_0) \to^* (v_2, \sigma_2)$, and $v = (v_1, v_2)$, with $\sigma' = \sigma_1 \cup \sigma_2$ by facts (a,b). The IH applies to each subterm (genuine sub-derivations): v_i is τ_i-shaped, $\mathsf{out}(C(e_i)[\rho_{\gamma_i}]) = \mathsf{flat}_{\tau_i}(v_i)$, and $\mathsf{portout}(C(e_i)[\rho_{\gamma_i}]) = \sigma_i$. Hence v is $(\tau_1 * \tau_2)$-shaped, and since C-Pair unions the two subgraphs (concatenating outputs, without new $\mathtt{output}(p)$ nodes), $\mathsf{out}(C(e)[\rho_\gamma]) = \mathsf{flat}_{\tau_1}(v_1) \cdot \mathsf{flat}_{\tau_2}(v_2) = \mathsf{flat}_\tau(v)$ and $\mathsf{portout}(C(e)[\rho_\gamma]) = \sigma_1 \cup \sigma_2 = \sigma'$.

Cases (T-Phase), (T-Split), (T-Unitary). These operate on one or two optical subterms and write no port. Take T-Split: $(e_0\gamma, \sigma_0) \to^* (v_0, \sigma')$ and then $v = (\mathsf{amp}(v_0, r), \mathsf{amp}(v_0, 1-r))$. The IH on e_0 gives v_0 Opt-shaped, $\mathsf{out}(C(e_0)[\rho_\gamma]) = (v_0)$, and $\mathsf{portout} = \sigma'$. C-Split appends a splitter and two valued outputs carrying $\mathsf{amp}(v_0, r)$ and $\mathsf{amp}(v_0, 1-r)$, so v is (Opt $*$ Opt)-shaped, $\mathsf{out}(C(e)[\rho_\gamma]) = \mathsf{flat}_{\mathsf{Opt}*\mathsf{Opt}}(v)$, and, adding no $\mathtt{output}(p)$ node, portout is unchanged $= \sigma'$. T-Phase and T-Unitary are identical, using $v = \mathsf{phase}(\theta, v_0)$ and the two components of $U \cdot [v_1; v_2]$ respectively.

Case (T-Out). $e = \mathtt{output}(p) \leftarrow e_1;\, e_2$, $\Delta = \Delta_1, \Delta_2$, $\gamma = \gamma_1 \uplus \gamma_2$. Evaluation gives $(e_1\gamma_1, \sigma_0) \to^* (v_1, \sigma_1)$, then a write of p, then $(e_2\gamma_2, \sigma_0) \to^* (v, \sigma_2)$; by facts (a,b) the domains σ_1, $\{p\}$, σ_2 are disjoint and $\sigma' = \sigma_1 \cup \{p \mapsto v_1\} \cup \sigma_2$. By IH, $\mathsf{out}(C(e_1)[\rho_{\gamma_1}]) = (v_1)$ with portout $= \sigma_1$, and $\mathsf{out}(C(e_2)[\rho_{\gamma_2}]) = \mathsf{flat}_\tau(v)$ with portout $= \sigma_2$; also v is τ-shaped. C-Out keeps the expression outputs of e_2 and adds one $\mathtt{output}(p)$ node fed by the unique output of $C(e_1)[\rho_{\gamma_1}]$, whose value is v_1. Hence $\mathsf{out}(C(e)[\rho_\gamma]) = \mathsf{flat}_\tau(v)$, and the $\mathtt{output}$ nodes contribute σ_1 (from e_1), $\{p \mapsto v_1\}$ (the new node), and σ_2 (from e_2), so $\mathsf{portout}(C(e)[\rho_\gamma]) = \sigma'$.

Case (T-LetLin). $e = \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2$ with premises $\Delta_1; \Pi_1 \vdash e_1 : \tau_1$ and $\Delta_2, x : \tau_1; \Pi_2 \vdash e_2 : \tau$; split $\gamma = \gamma_1 \uplus \gamma_2$. Then $e\gamma = \mathtt{let}\ x = e_1\gamma_1\ \mathtt{in}\ e_2\gamma_2$, whose evaluation decomposes (reduce $e_1\gamma_1$, then E-Let2) as

$$(e_1\gamma_1, \sigma_0) \to^* (v_1, \sigma_1), \qquad (e_2\gamma_2[x \mapsto v_1], \sigma_1) \to^* (v, \sigma'),$$

and $e_2\gamma_2[x \mapsto v_1] = e_2\gamma_2'$ where $\gamma_2' = \gamma_2 \uplus [x \mapsto v_1]$. The IH on e_1 (a sub-derivation) under γ_1 gives that v_1 is τ_1-shaped, and, writing $H_1 = C(e_1)[\rho_{\gamma_1}]$, $\mathsf{out}(H_1) = \mathsf{flat}_{\tau_1}(v_1)$ and $\mathsf{portout}(H_1) = \sigma_1$. Since v_1 is τ_1-shaped, γ_2' is a legal environment for e_2; the IH on e_2 (a sub-derivation) under γ_2' gives that v is τ-shaped, $\mathsf{out}(C(e_2)[\rho_{\gamma_2'}]) = \mathsf{flat}_\tau(v)$, and $\mathsf{portout}(C(e_2)[\rho_{\gamma_2'}]) = \sigma_2$, where σ_2 is the write-set of $e_2\gamma_2'$; by facts (a,b), $\sigma' = \sigma_1 \cup \sigma_2$. Note the induction hypothesis is applied only to the subterms e_1 and e_2; the value v_1 enters through the environment, not through the term.

Now assemble the graph. By C-Let, $C(e) = C(e_2)[x \mapsto C(e_1)]$. The substitutions of ρ_γ act on the disjoint placeholder sets of Δ_1 (inside the $C(e_1)$-part) and Δ_2 (inside $C(e_2)$), so they commute with the x-splice and

$$C(e)[\rho_\gamma] = B[x \mapsto H_1], \qquad B = C(e_2)[\rho_{\gamma_2}], \quad H_1 = C(e_1)[\rho_{\gamma_1}].$$

By fact (c), $\mathrm{out}(C(v_1)) = \mathrm{flat}_{\tau_1}(v_1) = \mathrm{out}(H_1)$. Splice invariance (Lemma D.7) then gives $\mathrm{out}(B[x \mapsto H_1]) = \mathrm{out}(B[x \mapsto C(v_1)])$, with the two graphs agreeing on every $\mathtt{output}(p)$ node inherited from B. But $B[x \mapsto C(v_1)] = C(e_2)[\rho_{\gamma_2}, x \mapsto C(v_1)] = C(e_2)[\rho_{\gamma_2'}]$. Therefore: *(i)* $\mathrm{out}(C(e)[\rho_\gamma]) = \mathrm{out}(C(e_2)[\rho_{\gamma_2'}]) = \mathrm{flat}_\tau(v)$.

(ii) The $\mathtt{output}(p)$ nodes of $B[x \mapsto H_1]$ split into those inside H_1—which the single splice keeps intact together with their predecessor values, contributing $\mathrm{portout}(H_1) = \sigma_1$—and those inherited from B, whose predecessor values match those in $C(e_2)[\rho_{\gamma_2'}]$ by splice invariance, contributing σ_2. By port-linearity these domains are disjoint, so $\mathrm{portout}(C(e)[\rho_\gamma]) = \sigma_1 \cup \sigma_2 = \sigma'$.

Case (T-UnpackLin). $e = \mathtt{unpack}\ e_0\ \mathtt{as}\ (y_1, y_2)\ \mathtt{in}\ e_2$ with premises $\Delta_0; \Pi_0 \vdash e_0 : \tau_1 * \tau_2$ and $\Delta_2, y_1 : \tau_1, y_2 : \tau_2; \Pi_2 \vdash e_2 : \tau$; split $\gamma = \gamma_0 \uplus \gamma_2$. Evaluation of $e\gamma$ decomposes (reduce $e_0\gamma_0$, then E-Unpack2) as

$$(e_0\gamma_0, \sigma_0) \to^* ((v_1, v_2), \sigma_1), \qquad (e_2\gamma_2[y_1 \mapsto v_1, y_2 \mapsto v_2], \sigma_1) \to^* (v, \sigma'),$$

and the continuation is $e_2\gamma_2''$ with $\gamma_2'' = \gamma_2 \uplus [y_1 \mapsto v_1, y_2 \mapsto v_2]$. The IH on e_0 under γ_0 gives that (v_1, v_2) is $(\tau_1 * \tau_2)$-shaped—so each v_i is τ_i-shaped—and, writing $G_0^\gamma = C(e_0)[\rho_{\gamma_0}]$, $\mathrm{out}(G_0^\gamma) = \mathrm{flat}_{\tau_1}(v_1) \cdot \mathrm{flat}_{\tau_2}(v_2)$ with $\mathrm{portout}(G_0^\gamma) = \sigma_1$. As v_1, v_2 are shaped, γ_2'' is legal for e_2; the IH on e_2 under γ_2'' gives v τ-shaped, $\mathrm{out}(C(e_2)[\rho_{\gamma_2''}]) = \mathrm{flat}_\tau(v)$, and $\mathrm{portout}(C(e_2)[\rho_{\gamma_2''}]) = \sigma_2$, with $\sigma' = \sigma_1 \cup \sigma_2$. Again the IH touches only the subterms.

By C-Unpack, $C(e) = C(e_2)[(y_1, y_2) \mapsto C(e_0)]$, and as in the let case the outer substitutions commute with the (y_1, y_2)-splice, so $C(e)[\rho_\gamma] = B[(y_1, y_2) \mapsto G_0^\gamma]$ with $B = C(e_2)[\rho_{\gamma_2}]$. Let W be the value graph obtained by placing $C(v_1)$ and $C(v_2)$ side by side, ordering their outputs as block 1 then block 2; by fact (c), $\mathrm{out}(W) = \mathrm{flat}_{\tau_1}(v_1) \cdot \mathrm{flat}_{\tau_2}(v_2) = \mathrm{out}(G_0^\gamma)$. Splice invariance gives $\mathrm{out}(B[(y_1, y_2) \mapsto G_0^\gamma]) = \mathrm{out}(B[(y_1, y_2) \mapsto W])$, with agreement on all $\mathtt{output}(p)$ nodes inherited from B. As W is a disjoint union with the block ordering, the combined splice into W equals the two single splices $B[y_1 \mapsto C(v_1)][y_2 \mapsto C(v_2)] = C(e_2)[\rho_{\gamma_2''}]$. Therefore:

- *(i)* $\mathrm{out}(C(e)[\rho_\gamma]) = \mathrm{out}(C(e_2)[\rho_{\gamma_2''}]) = \mathrm{flat}_\tau(v)$.
- *(ii)* The $\mathtt{output}(p)$ nodes split into those inside G_0^γ—kept intact by the single splice and contributing $\mathrm{portout}(G_0^\gamma) = \sigma_1$—and those inherited from B, contributing σ_2 by splice invariance. Being disjoint (port-linearity), they give $\mathrm{portout}(C(e)[\rho_\gamma]) = \sigma_1 \cup \sigma_2 = \sigma'$.

This case is exactly where splicing the *whole* of $C(e_0)$ once matters: had we spliced only the two output blocks, the writes σ_1 performed inside e_0 would have been dropped.

This exhausts the typing rules, completing the induction. □

Theorem D.16 (Compilation Preserves Expression Outputs). *Let $\emptyset; \Pi \vdash e : \tau$ and for any input environment ι such that $\iota(p)$ is defined for every port p in Π, let $\sigma_0 = \emptyset$. If $(e, \sigma_0) \to_\iota^* (v, \sigma')$ and $C(e) = (V, E, \ell, \Sigma, O, t, r, \mathrm{Val})$, then*

$$\mathrm{out}\big(C(e)\big) = \mathrm{flat}_\tau(v).$$

Proof. This is the empty-substitution instance of Lemma D.15: take $\Delta = \emptyset$ and γ empty, so that $e\gamma = e$ and $C(e)[\rho_\gamma] = C(e)$. Part (i) of that lemma then reads $\mathrm{out}(C(e)) = \mathrm{flat}_\tau(v)$. □

Theorem D.17 (Compilation Preserves Port Outputs). *Let $\emptyset; \Pi \vdash e : \tau$ and for any input environment ι such that $\iota(p)$ is defined for every port p in Π. Let $\sigma_0 = \emptyset$. If $(e, \sigma_0) \to_\iota^* (v, \sigma')$ and $C(e) = (V, E, \ell, \Sigma, O, t, r, \mathrm{Val})$, then*

$$\mathrm{portout}\big(C(e)\big) = \sigma'.$$

Proof. This is again the empty-substitution instance of Lemma D.15: with $\Delta = \emptyset$ and γ empty, $C(e)[\rho_\gamma] = C(e)$, and part (ii) of the lemma reads $\mathrm{portout}(C(e)) = \sigma'$. □

E NP-Completeness of the Graph Embedding

We first formally define the Constrained Graph Embedding (CGE) problem. An instance consists of a graph $G = (V, E, l)$, a host graph $H = (U, A)$ with side groups $\{S_{h,s}\}$. Let $\bar{A}$ denote the set of undirected links in the host: $\{i, j\} \in \bar{A}$ iff $(i, j) \in A$ or $(j, i) \in A$. We ask whether there exist: (i) an injective placement $\pi : V \to U$ such that $l(v) \in T(\pi(v))$ for all $v \in V$, and (ii) for each logical edge $e = (u, v) \in E$, a simple directed path P_e in H from $\pi(u)$ to $\pi(v)$, such that:

- (edge-disjointness) for every $\{i, j\} \in \bar{A}$, at most one path P_e uses an arc in $\{(i, j), (j, i)\}$,
- (no spurious components) no internal vertex of any P_e lies in $\pi(V)$, and
- (side constraints) If $\pi(v) = h$, then every incoming path to v enters h from the same side group, and
- (no same-side V-turns) for any path P_e and any internal host h on P_e with predecessor p and successor q, there is no side group $S_{h,s}$ with $p, q \in S_{h,s}$.

The Constrained Graph Embedding (CGE) is NP-complete:

Theorem 4.1. *CGE is NP-complete.*

Proof. **Membership in NP.** A certificate consists of a placement $\pi : V \to U$ and paths $\{P_e : e \in E\}$. All constraints (injectivity, label compatibility, path validity, edge-disjointness, no spurious components, side constraints, no V-turns) can be verified in polynomial time.

NP-hardness. We reduce from the edge-disjoint paths (EDP) problem on undirected graphs, which is NP-complete [52]. An EDP instance is an undirected graph $H_0 = (U_0, \bar{A}_0)$ with k terminal pairs (s_i, t_i). The question is whether there exist k pairwise edge-disjoint paths connecting each s_i to t_i.

As standard preprocessing, for each terminal $x \in \{s_i, t_i\}$, add a new leaf x' adjacent only to x, and replace x by x' as the terminal. This preserves feasibility and ensures every terminal has degree 1.

Construction. *Host graph H:* Bidirect H_0 by replacing each edge $\{p, q\} \in \bar{A}_0$ with arcs (p, q) and (q, p).

Side groups: For each $h \in U$, define singleton side groups: $S_{h,p} = \{p\}$ for each neighbor p of h.

Component graph G: Create k isolated edges: $V = \{a_1, b_1, \ldots, a_k, b_k\}$ and $E = \{e_i = (a_i, b_i) : i = 1, \ldots, k\}$.

Labels: Introduce distinct labels $\lambda_{s_i}, \lambda_{t_i}$ for each i. Set $l(a_i) = \lambda_{s_i}$ and $l(b_i) = \lambda_{t_i}$. Define $T(s_i) = \{\lambda_{s_i}\}$, $T(t_i) = \{\lambda_{t_i}\}$, and $T(u) = \emptyset$ for all other $u \in U$. This forces $\pi(a_i) = s_i$ and $\pi(b_i) = t_i$.

First, suppose EDP has solution $Q_1, \ldots, Q_k$ (edge-disjoint paths in H_0). We may assume each Q_i is simple (remove any cycles).

Orient each Q_i from s_i to t_i to get directed path P_{e_i} in H. Define $\pi(a_i) = s_i$, $\pi(b_i) = t_i$. We check:

- π is injective (all terminals distinct) and label-compatible (by construction).
- Each P_{e_i} is a simple directed path from s_i to t_i.
- Edge-disjointness in H_0 implies edge-disjointness in H (at most one Q_i uses each undirected edge).
- No spurious components: Each terminal has degree 1, so it cannot be an internal vertex (would need degree ≥ 2).
- Side constraints: Each vertex in G has in-degree ≤ 1, so at most one incoming path. Constraint is vacuous.
- No V-turns: Side groups are singletons, so p and q can share a side group only if $p = q$, but simple paths have distinct consecutive vertices.

In the other direction, suppose CGE has solution $(\pi, \{P_{e_i}\})$. Label compatibility forces $\pi(a_i) = s_i$ and $\pi(b_i) = t_i$. Thus each P_{e_i} is a directed path from s_i to t_i in H.

Forget arc orientations to get undirected paths Q_i in H_0. CGE edge-disjointness ensures at most one path uses each arc from $\{(p, q), (q, p)\}$, hence at most one undirected path uses edge $\{p, q\}$. Therefore $Q_1, \ldots, Q_k$ are edge-disjoint in H_0.

Conclusion. The reduction is polynomial-time and preserves feasibility in both directions. Since EDP is NP-complete, CGE is NP-hard. Combined with NP membership, CGE is NP-complete. □

F Unitary Inference Algorithm

We present the full algorithm for inferring a unitary matrix from a linear specification in Figure 24. The objective of the algorithm is to embed a $k \times n$ linear specification matrix R into a square unitary matrix $U \in \mathbb{C}^{N \times N}$. This process ensures that the desired linear transformation is physically realizable in a lossless system by utilizing a minimal number of ancilla modes.

Step 1: PSD Analysis and Ancilla Count The algorithm begins by assessing the feasibility of the specification. We define the matrix as $S = I_k - RR^*$.

- **Feasibility:** For a unitary realization to exist, R must be a contraction, meaning S must be positive semidefinite ($S \succeq 0$). This ensures the specification does not require power gain.
- **Minimality:** The number of required ancilla inputs m is determined by the rank of S.

Step 2: Constructing the Matrix To account for the signal not captured by R, we construct a supplementary matrix $A \in \mathbb{C}^{k \times m}$. By performing an eigendecomposition of the matrix $S = V\Lambda V^*$ and setting $A = V_+\sqrt{\Lambda_+}$, we ensure that $AA^* = S$. We then form the augmented matrix $E = [R \mid A]$. By construction, the rows of E are perfectly orthonormal ($EE^* = I_k$), representing the k constrained output rows of our final system.

Step 3: Orthonormal Completion via Nullspace Since U must be a square $N \times N$ matrix (where $N = n + m$), we must find $N - k$ additional rows that are mutually orthonormal and orthogonal to E.

(1) We compute a basis Z for the **nullspace** of E. These vectors represent the degrees of freedom in the system that do not contribute to the specified outputs.
(2) Because the nullspace basis is generally not orthonormal, we form the Gram matrix $G = Z^*Z$ and compute its Cholesky factorization LL^*.
(3) We define the complementary row block $F = (ZL^{-*})^*$. This ensures that F consists of orthonormal rows and that $FE^* = 0$.

Step 4: Assembly and Row Mapping The final unitary U is synthesized by vertically stacking the constrained block E and the complementary block F. To satisfy the physical layout of the system, a row permutation is applied such that the rows of E (containing the specification R) are mapped to the user-defined target output indices $\mathcal{I}$. The resulting matrix U satisfies the identity $U[z; \mathbf{0}_m] = [y; *]$, where the k designated outputs match the specification $y = Rz$ exactly. We now prove the correctness of the algorithm through a series of lemmas and theorems. First we show that a matrix R can be embedded into a unitary if and only if $I_k - RR^*$ is PSD, and that the minimal number of ancilla modes required is given by the rank of this matrix.

Lemma F.1 (Minimal Ancilla Feasibility). *A linear specification $R \in \mathbb{C}^{k \times n}$ can be embedded into a unitary matrix if and only if $I_k - RR^*$ is PSD. The minimal number of ancilla modes required is $m = \text{rank}(I_k - RR^*)$.*

Proof. Suppose there exists a unitary U with a sub-block $E = [R \mid A]$. Since the rows of a unitary matrix must be orthonormal, $EE^* = I_k$. Expanding this gives $RR^* + AA^* = I_k$, or $AA^* = I_k - RR^*$. Let $S = I_k - RR^*$. Because AA^* is Hermitian and positive semidefinite, $S \succeq 0$. The rank of S is $\text{rank}(AA^*) \leq \text{rank}(A) \leq m$.

Conversely, if $S \succeq 0$, we decompose $S = AA^*$ where A has $m = \text{rank}(S)$ columns. Thus $E = [R \mid A]$ satisfies $EE^* = RR^* + S = I_k$, completing the orthonormal set of rows. □

Next, we prove that the matrix U constructed by the algorithm is indeed unitary and satisfies the desired specification.

Theorem F.2 (Unitarity of Completion). *The matrix $U_0 = \begin{bmatrix} E \\ F \end{bmatrix}$ constructed by Algorithm 1 is unitary.*

Proof. We verify $U_0U_0^* = I_N$ via block multiplication:

$$U_0U_0^* = \begin{bmatrix} E \\ F \end{bmatrix} \begin{bmatrix} E^* & F^* \end{bmatrix} = \begin{bmatrix} EE^* & EF^* \\ FE^* & FF^* \end{bmatrix}$$

By construction $EE^* = I_k$. Furthermore, $FE^* = L^{-1}Z^*E^* = L^{-1}(EZ)^* = 0$ because $EZ = 0$. Finally, $FF^* = L^{-1}(Z^*Z)L^{-*} = L^{-1}(LL^*)L^{-*} = I_{N-k}$. Thus U_0 is unitary. □

Theorem F.3 (Satisfaction of Specification). *For any input $z \in \mathbb{C}^n$, the output vector $y = U[z; \mathbf{0}_m]$ satisfies $y_j = R_{j,\cdot}z$ for all $j \in \mathcal{I}$.*

Proof. Let $v = [z; \mathbf{0}_m]$. The j-th component of the output is $y_j = e_j^*Uv$. By construction, e_j^*U is the i-th row of $E = [R \mid A]$ for some i. Thus:

$$y_j = [R_{i,\cdot} \mid A_{i,\cdot}] \begin{bmatrix} z \\ \mathbf{0}_m \end{bmatrix} = R_{i,\cdot}z + A_{i,\cdot}\mathbf{0}_m = R_{i,\cdot}z.$$

The linear specification is exactly preserved while the ancilla components are nulled. □

Algorithm 1 Unitary Completion from Linear Specifications

Input: Specification matrix $R \in \mathbb{C}^{k \times n}$, target rows $\mathcal{I} \subset \{1, \ldots, N\}$
Step 1: PSD Analysis
Compute $S \leftarrow I_k - RR^*$
if $S \not\succeq 0$ **then**
    Error: Specification is unrealizable.
end if
$m \leftarrow \text{rank}(S)$
Compute eigendecomposition $S = V \Lambda V^*$
$A \leftarrow V_+ \sqrt{\Lambda_+}$ ▷ Using non-zero eigenvalues
$E \leftarrow [R \mid A] \in \mathbb{C}^{k \times (n+m)}$
Step 2: Orthonormal Complement
$N \leftarrow n + m$
Find $Z \in \mathbb{C}^{N \times (N-k)}$ s.t. $EZ = \mathbf{0}$ and $\text{rank}(Z) = N - k$
$G \leftarrow Z^* Z$
$L \leftarrow \text{cholesky}(G)$ ▷ $G = LL^*$
$F \leftarrow (ZL^{-*})^* \in \mathbb{C}^{(N-k) \times N}$
Step 3: Synthesis
$U_0 \leftarrow \begin{bmatrix} E \\ F \end{bmatrix}$
$U \leftarrow PU_0$ ▷ Permute rows to match indices $\mathcal{I}$
Output: Unitary $U \in \mathbb{C}^{N \times N}$, ancilla count m

Figure 24: Unitary Completion Algorithm